Pointer/Link List

A link list:

- is a one way list
- is a linear collection of data elements, called nodes

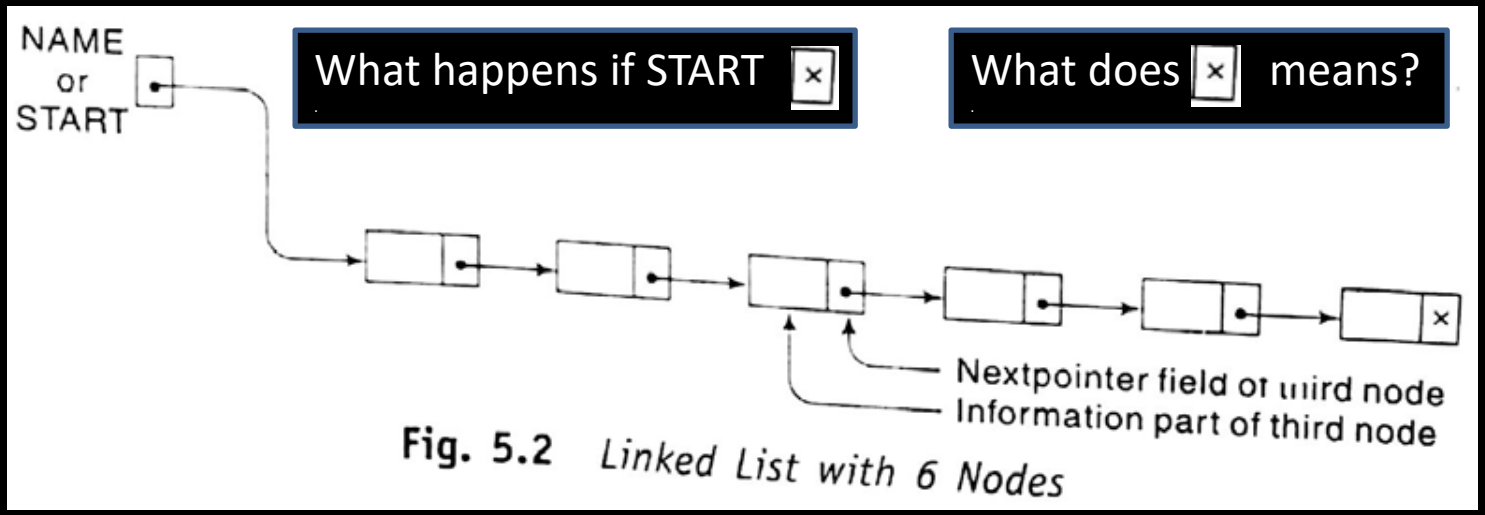


Fig. 5.2 *Linked List with 6 Nodes*

Where the linear order is given by means of pointers. Each node is divided into two parts:

- First part that contains information of element
- Second part that contains the address of next node. It is also known as link field, nextpointer field.
- The pointer of the last node contains a special value, called null pointer.
- There is a list pointer variable, called START, that contains the address of first node

CSE 203: Data Structure

Data Structure

Pointer /Link List

A hospital ward contains 12 beds, of which 9 are occupied as shown in Fig. 5.3. Suppose we want an alphabetical listing of the patients. **Start from node 5.**

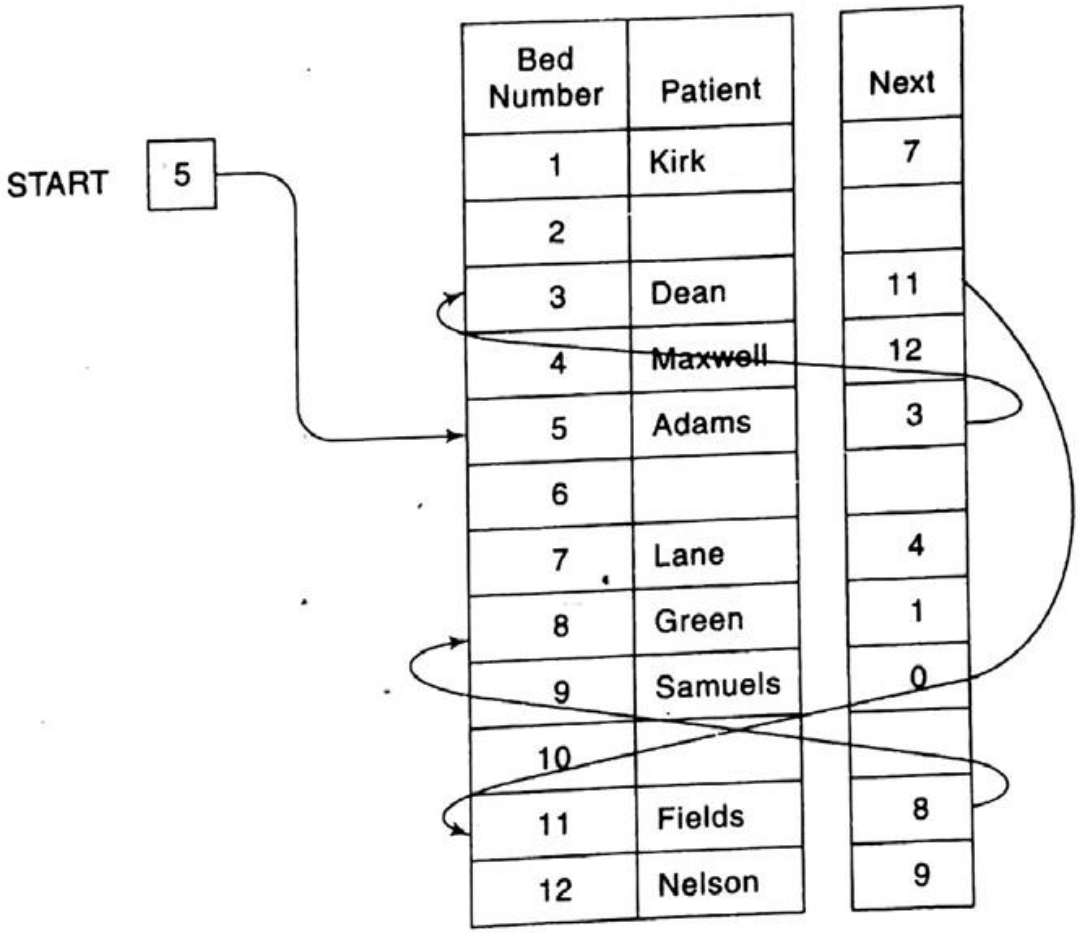


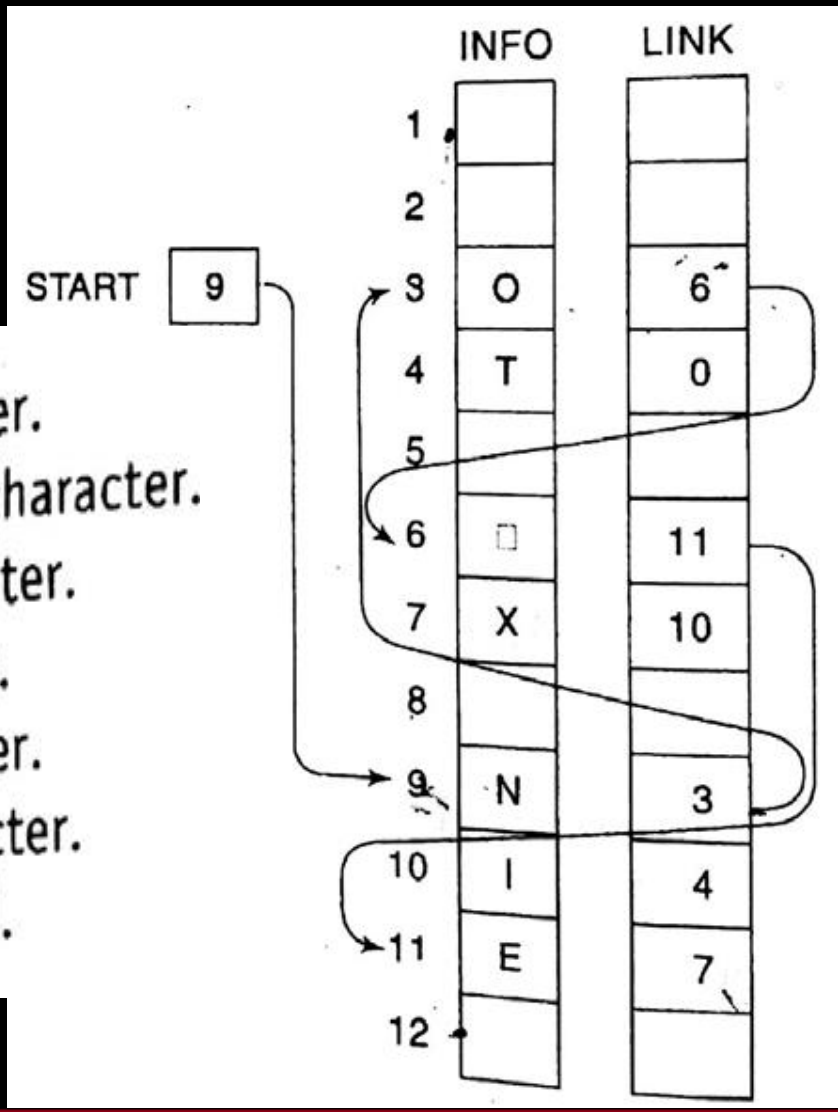
Fig. 5.3

Representation of Link List in memory

A link list for:

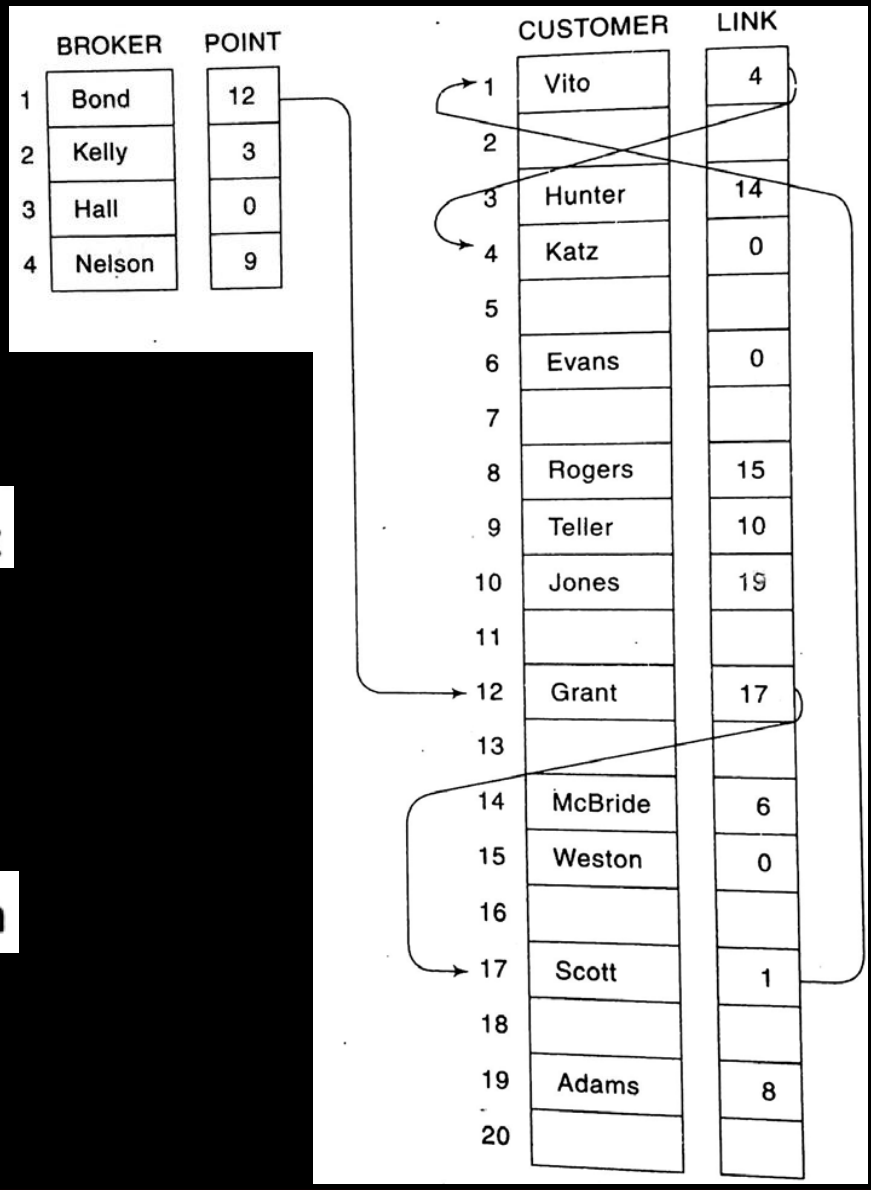
NO EXIT is the character string.

START = 9, so INFO[9] = N is the first character.
LINK[9] = 3, so INFO[3] = O is the second character.
LINK[3] = 6, so INFO[6] = □ (blank) is the third character.
LINK[6] = 11, so INFO[11] = E is the fourth character.
LINK[11] = 7, so INFO[7] = X is the fifth character.
LINK[7] = 10, so INFO[10] = I is the sixth character.
LINK[10] = 4, so INFO[4] = T is the seventh character.
LINK[4] = 0, the NULL value, so the list has ended.



CSE 203: Data Structure

Representation of Link List in memory



A link list for multiple start nodes:

Bond's list of customers: Grant, Scott, Vito, Katz

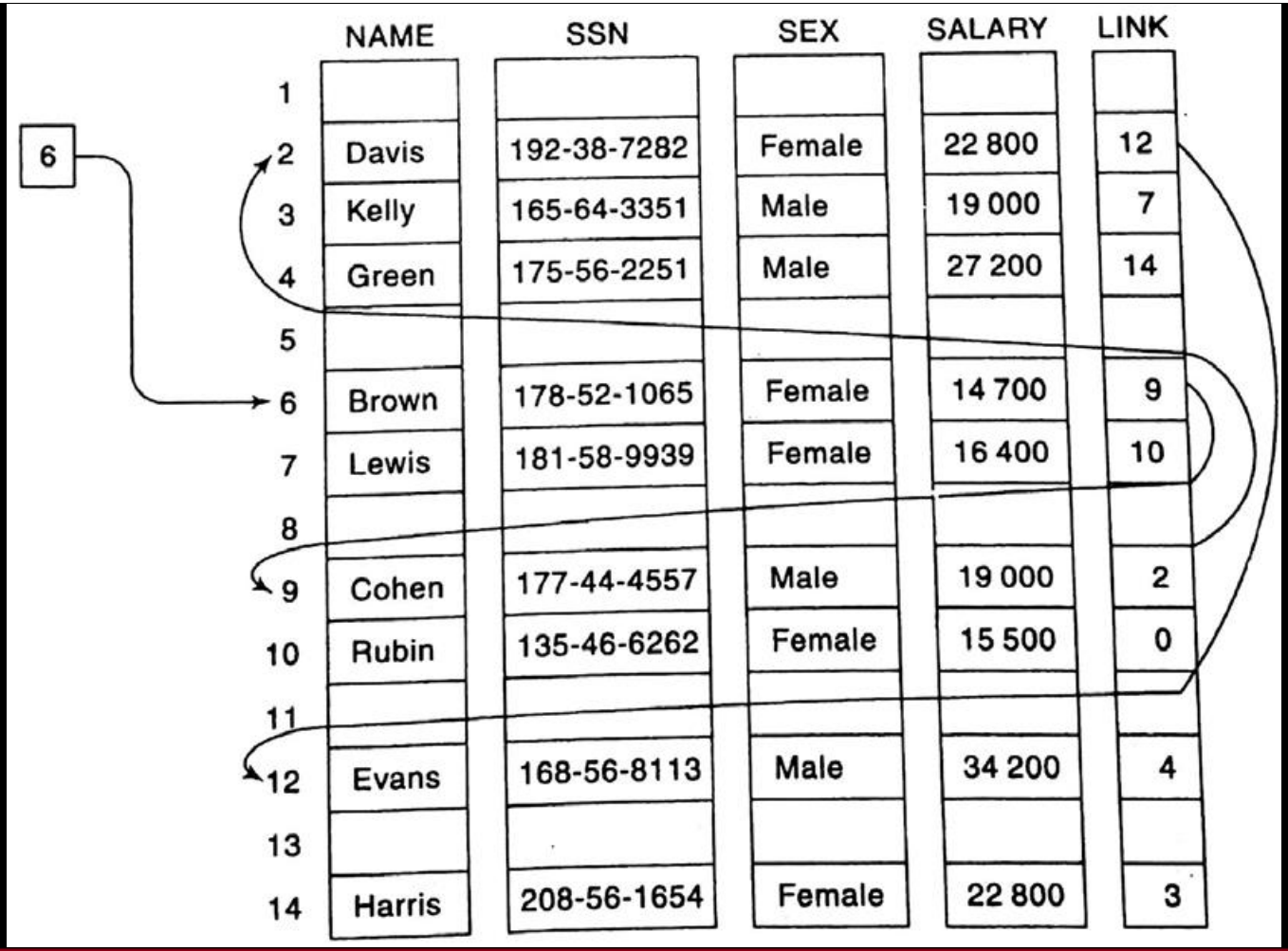
Kelly's list: Hunter, McBride, Evans

Nelson's list: Teller, Jones, Adams, Rogers, Weston

Hall's list is the null list

Representation of Link List in memory

Suppose the personnel file of a small company contains the following data on its nine employees: Name, Social Security Number, Sex, Monthly Salary Link list of multiple values



CSE 203: Data Structure

Data Structure

Traversing elements of a Link List

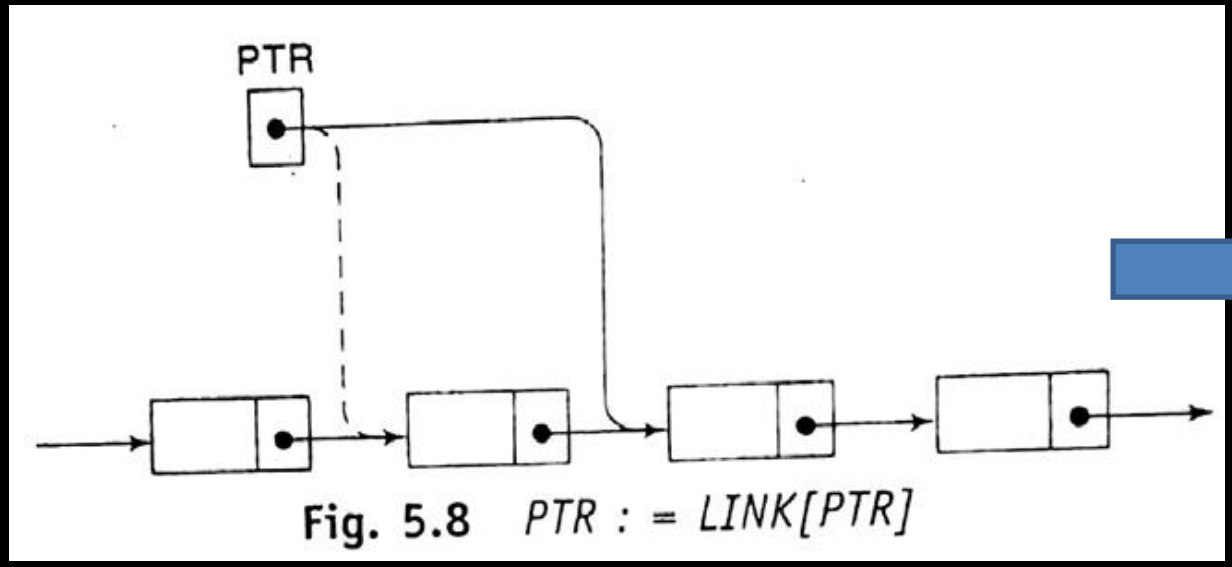
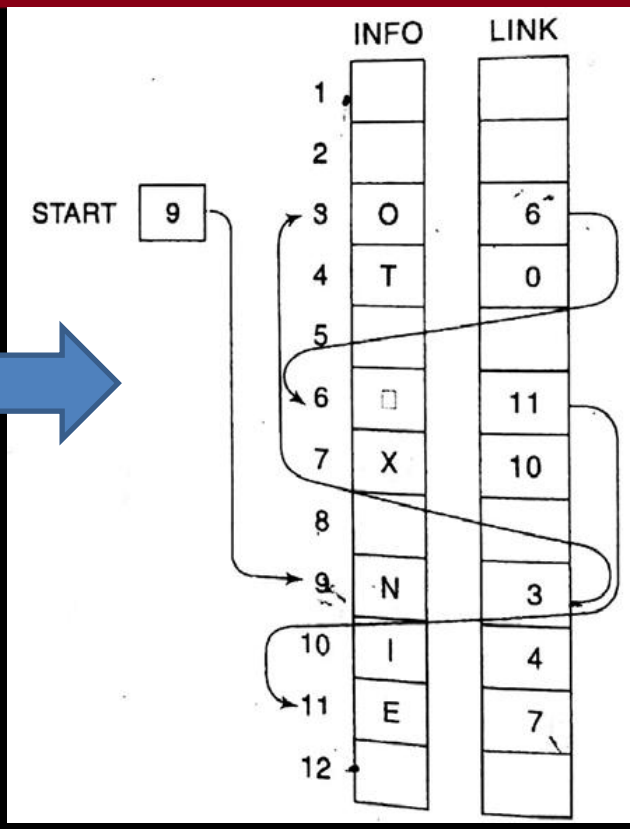


Fig. 5.8 *PTR : = LINK[PTR]*



update PTR by the assignment PTR := LINK[PTR], and then process INFO[PTR]

CSE 203: Data Structure

Data Structure

Traversing elements of a Link List

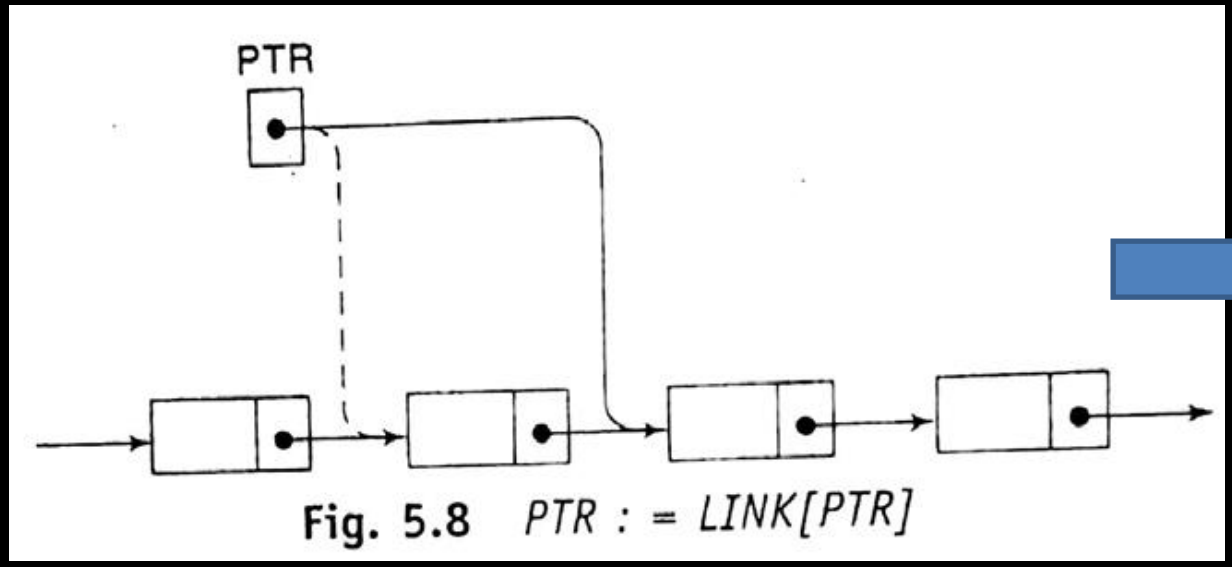
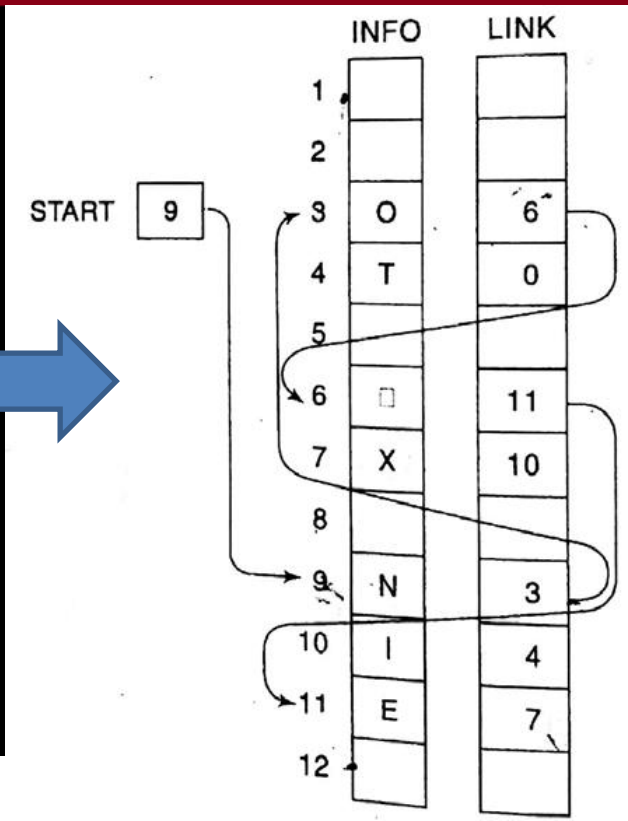


Fig. 5.8 *PTR : = LINK[PTR]*



1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR now points to the next node.]
   [End of Step 2 loop.]
5. Exit.

CSE 203: Data Structure

Data Structure

Traversing elements of a Link List

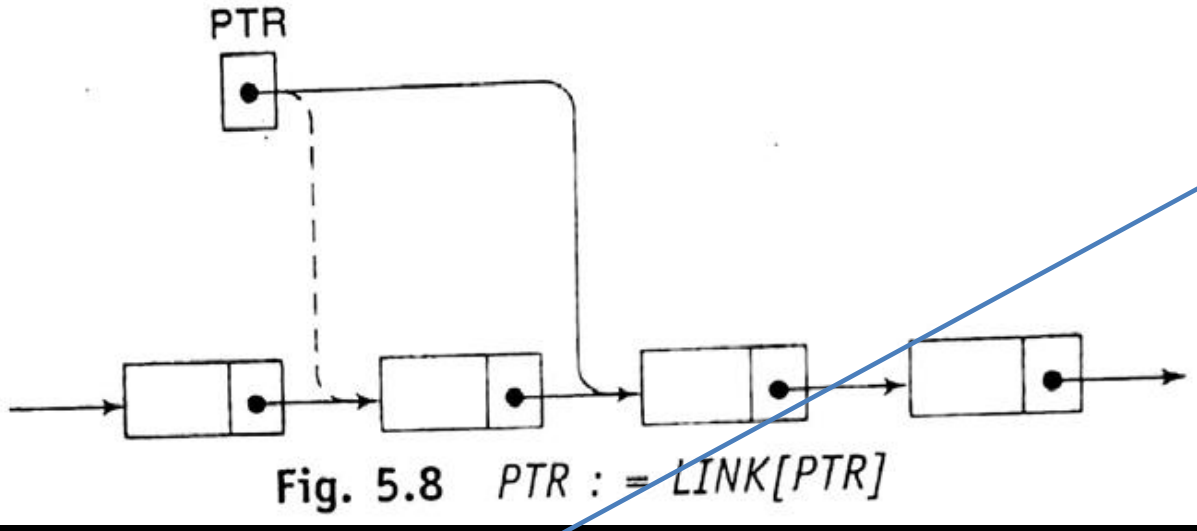
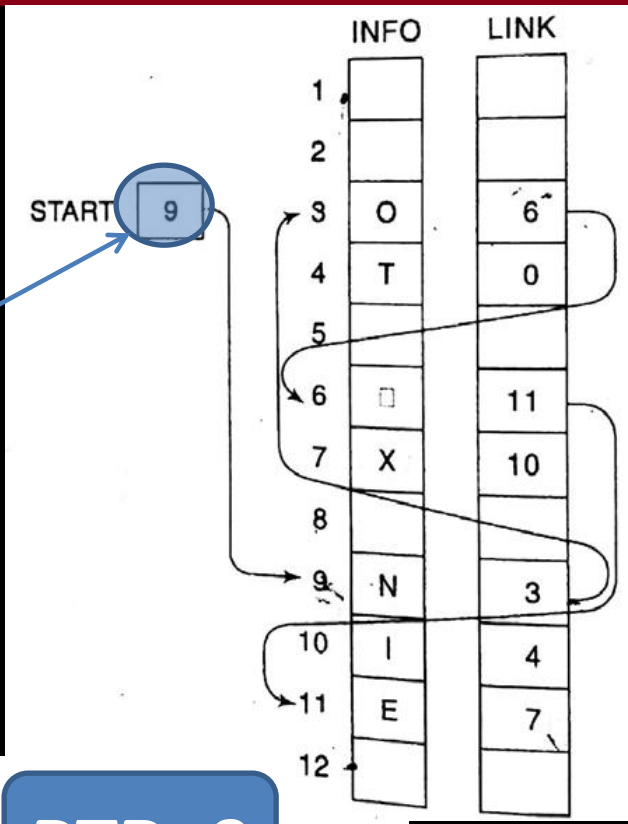


Fig. 5.8 *PTR : = LINK[PTR]*



1. Set PTR := START. [Initialize pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR now points to the next node.]
   [End of Step 2 loop.]
5. Exit.

While Executing

PTR=9

CSE 203: Data Structure

Data Structure

Traversing elements of a Link List

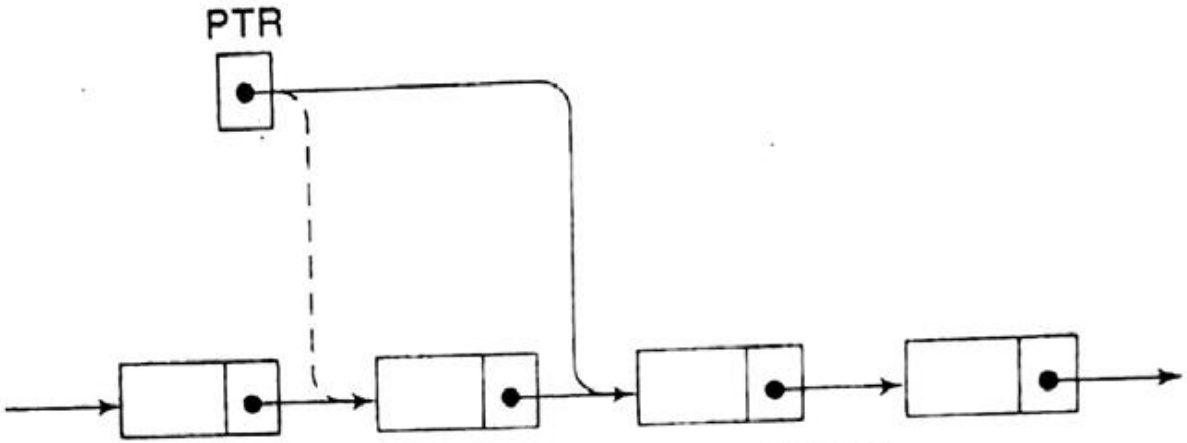
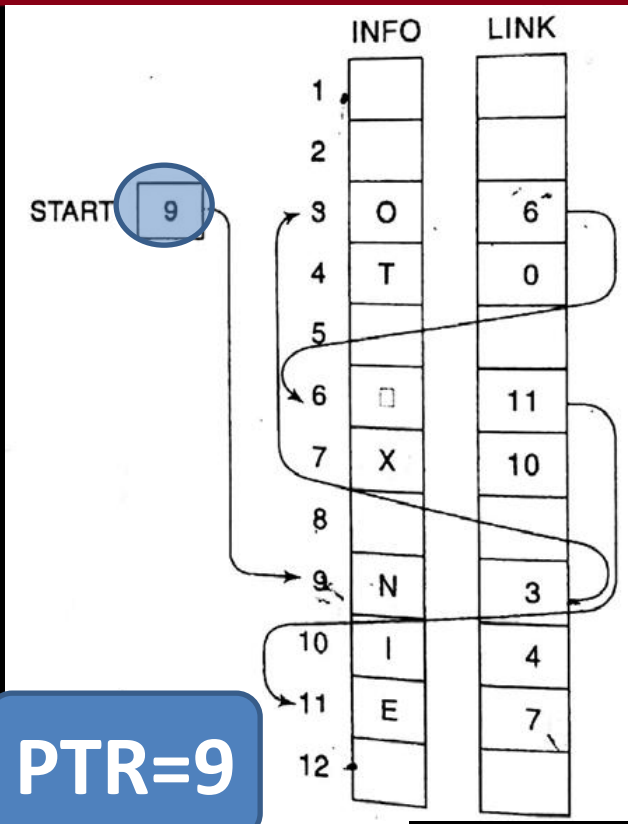


Fig. 5.8 *PTR : = LINK[PTR]*



PTR=9

1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR now points to the next node.]
   [End of Step 2 loop.]
5. Exit.

While Executing

CSE 203: Data Structure

Data Structure

Traversing elements of a Link List

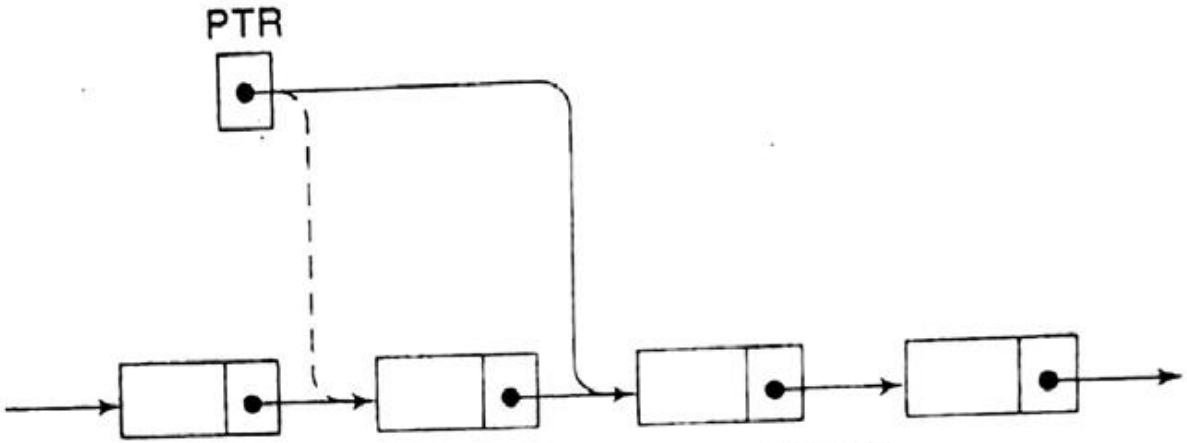
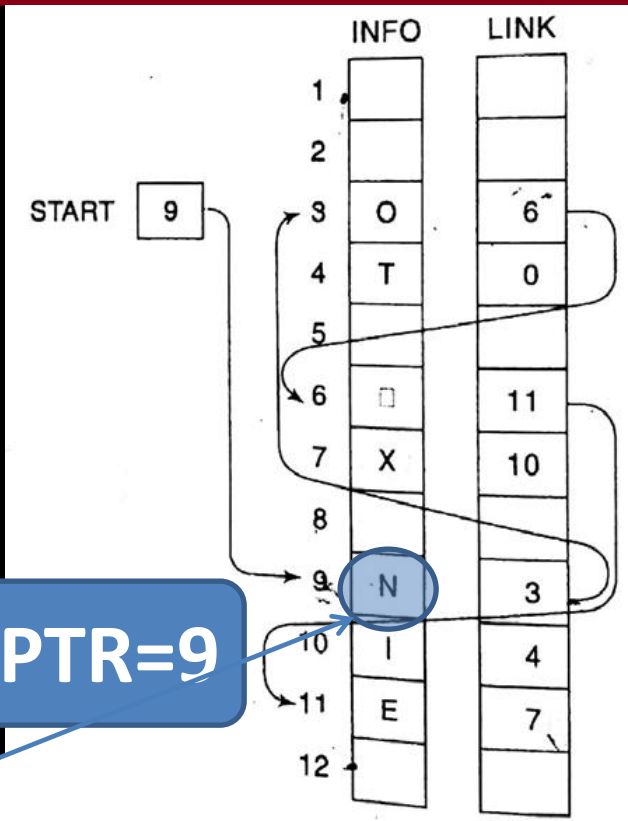


Fig. 5.8 *PTR : = LINK[PTR]*



PTR=9

1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR now points to the next node.]
   [End of Step 2 loop.]
5. Exit.

While Executing

CSE 203: Data Structure

Data Structure

Traversing elements of a Link List

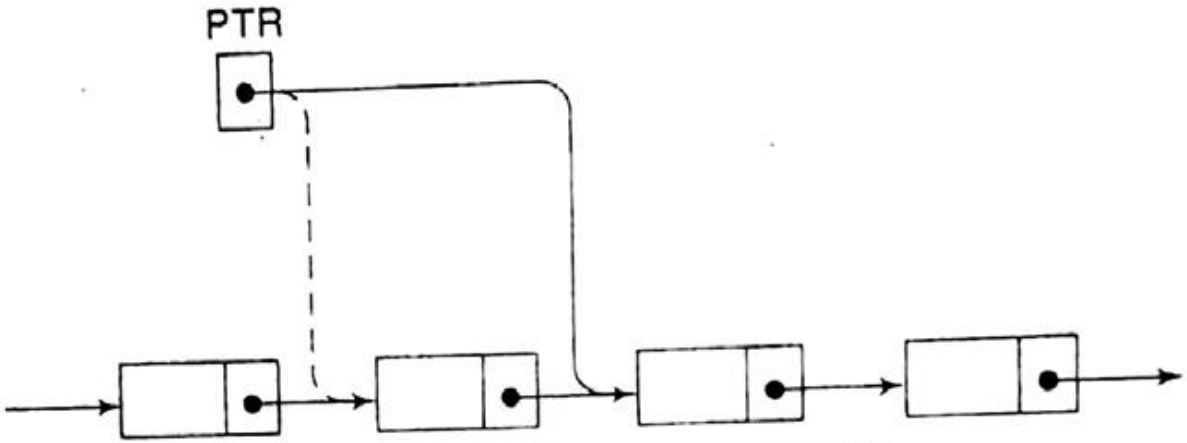
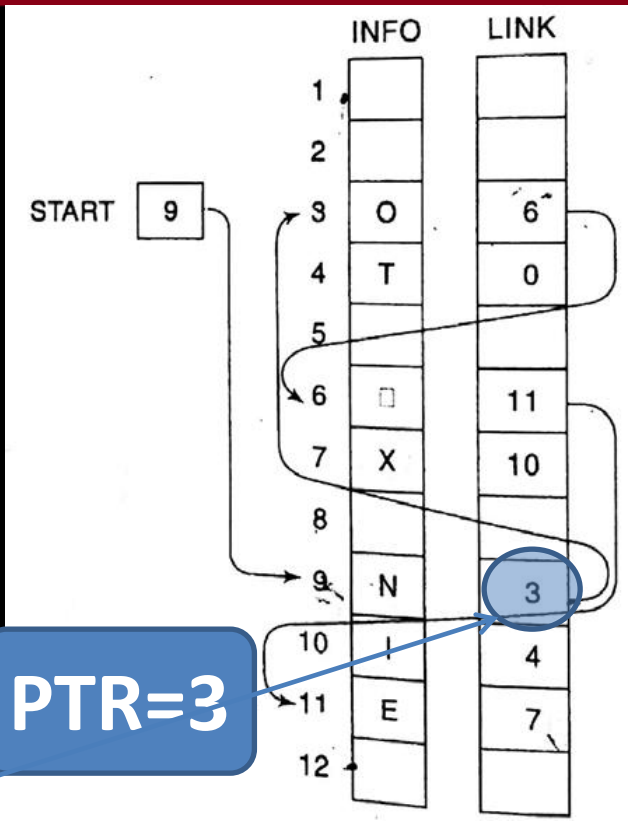


Fig. 5.8 *PTR : = LINK[PTR]*



PTR=3

1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR now points to the next node.]
   [End of Step 2 loop.]
5. Exit.

While Executing

CSE 203: Data Structure

Data Structure

Traversing elements of a Link List

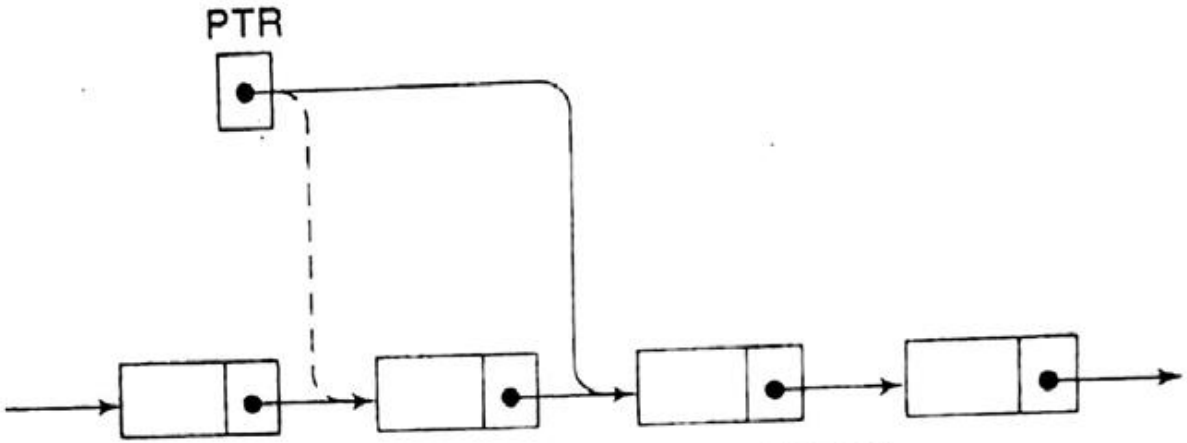
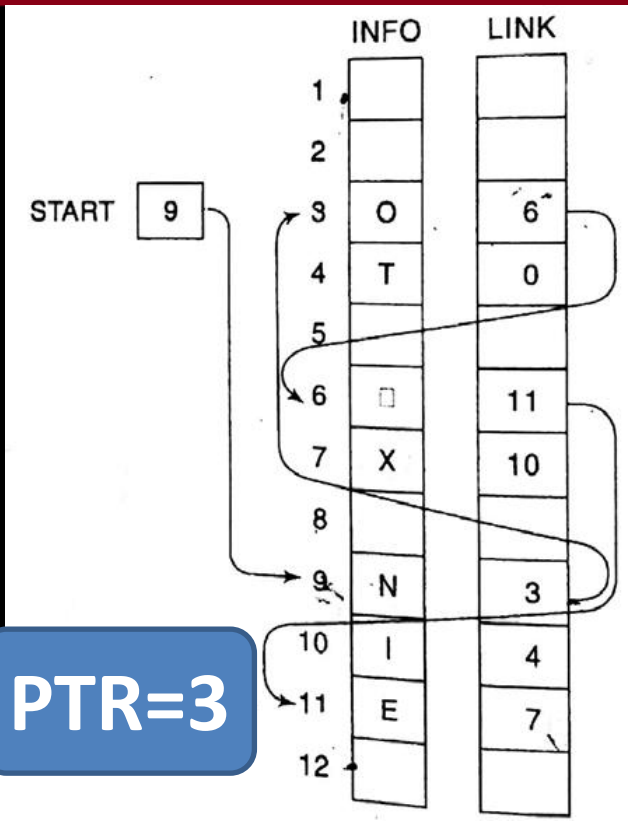


Fig. 5.8 *PTR : = LINK[PTR]*



PTR=3

1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR now points to the next node.]

[End of Step 2 loop.]

5. Exit.

While Executing

CSE 203: Data Structure

Data Structure

Traversing elements of a Link List

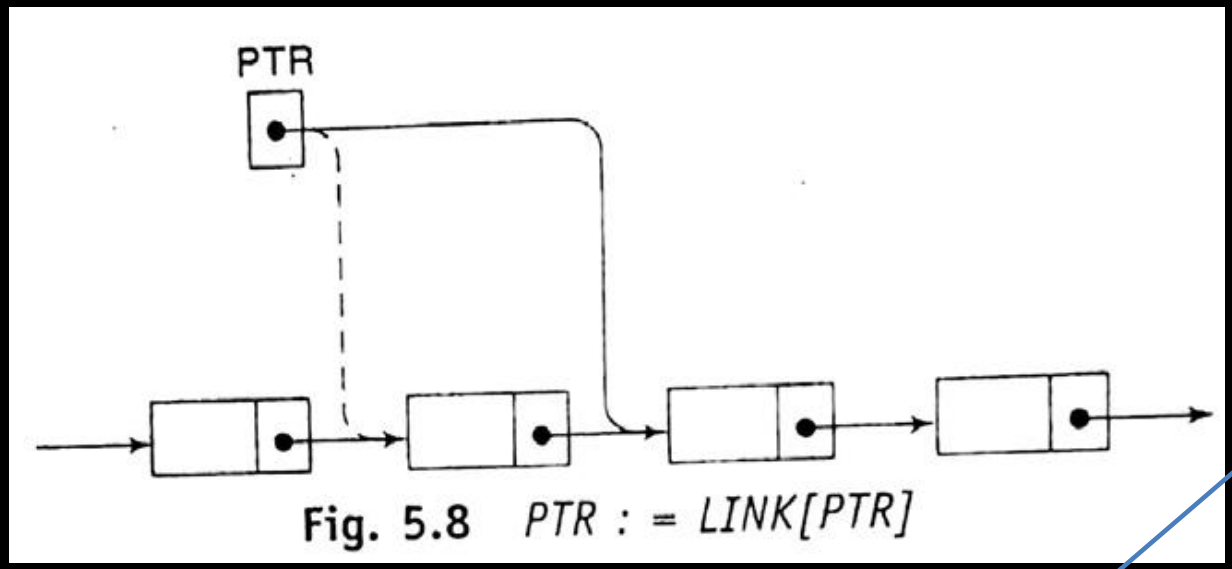
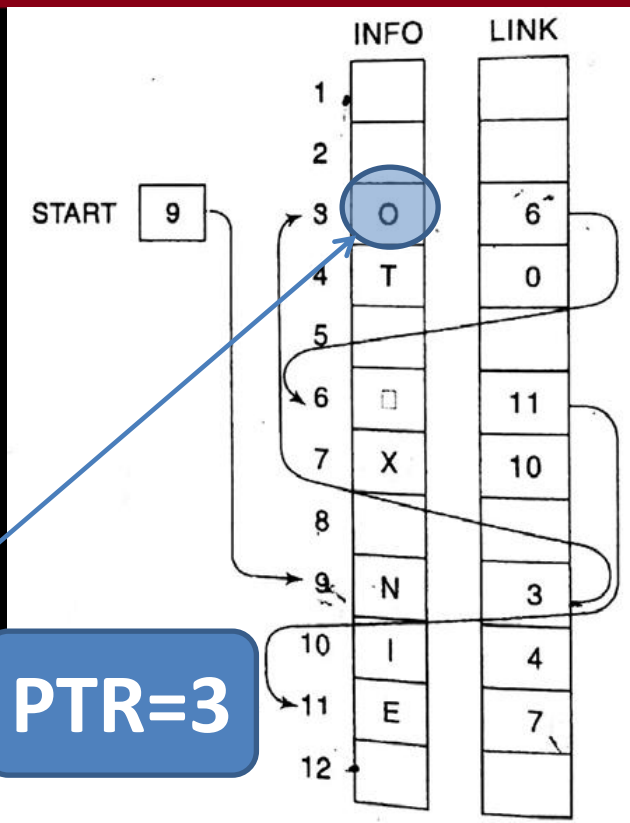


Fig. 5.8 *PTR : = LINK[PTR]*



PTR=3

1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR]. While Executing
4. Set PTR := LINK[PTR]. [PTR now points to the next node.]
   [End of Step 2 loop.]
5. Exit.

CSE 203: Data Structure

Data Structure

Traversing elements of a Link List

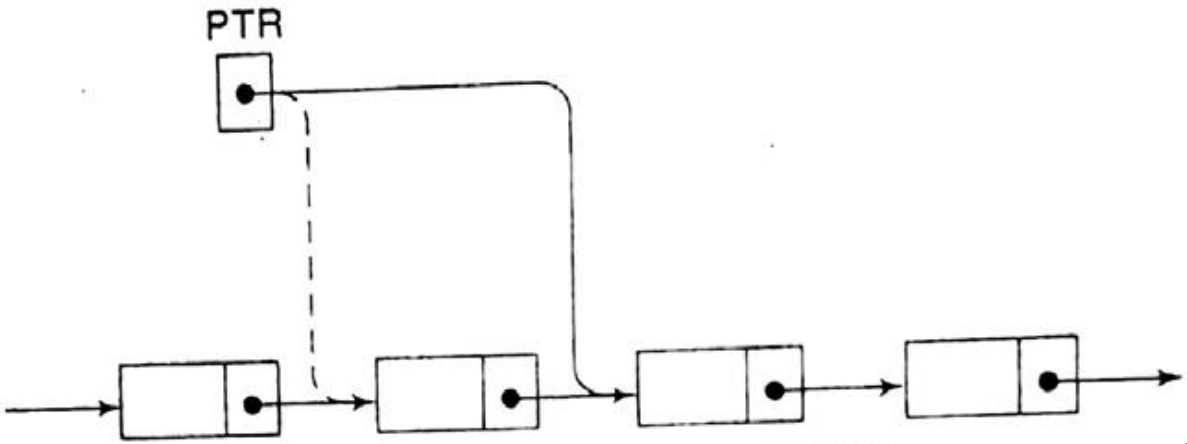
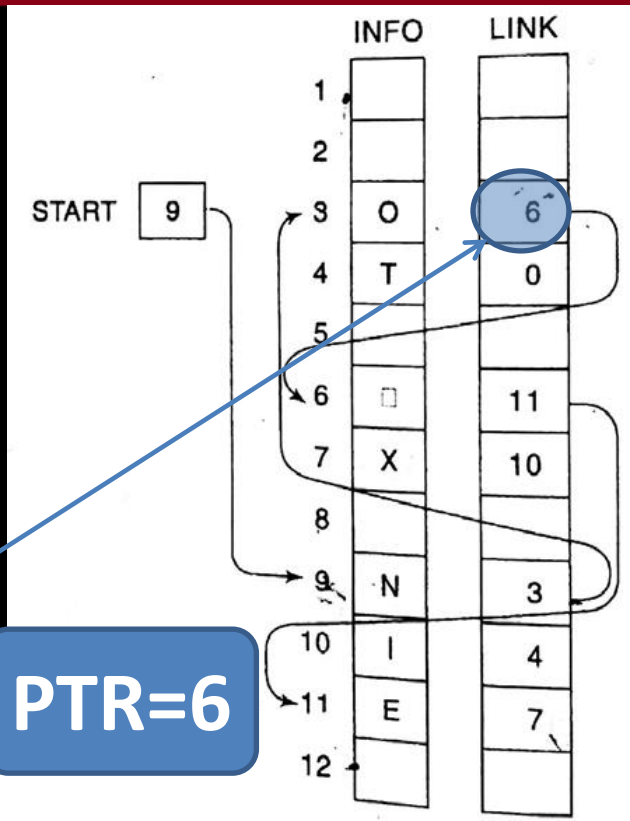


Fig. 5.8 *PTR : = LINK[PTR]*



PTR=6

1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR now points to the next node.]
   [End of Step 2 loop.]
5. Exit.

While Executing

CSE 203: Data Structure

Data Structure

Traversing elements of a Link List

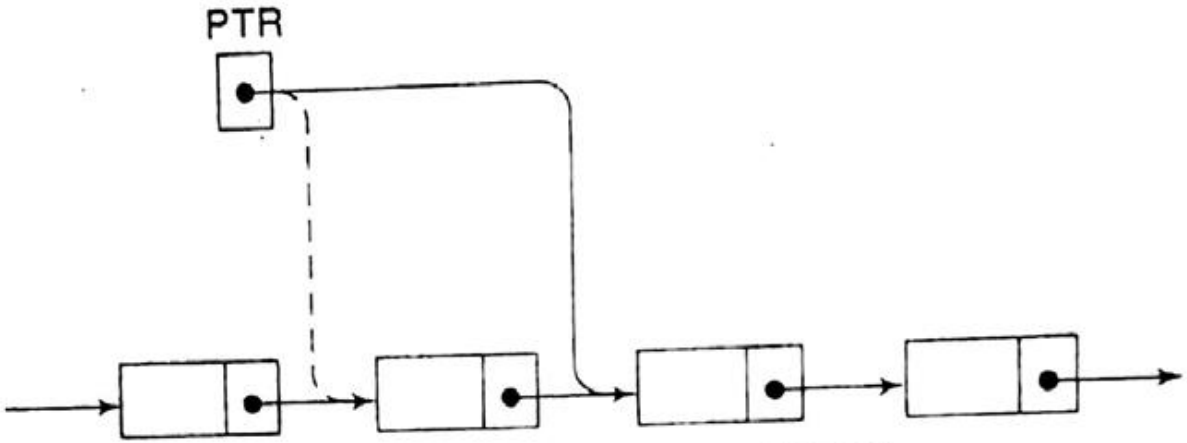
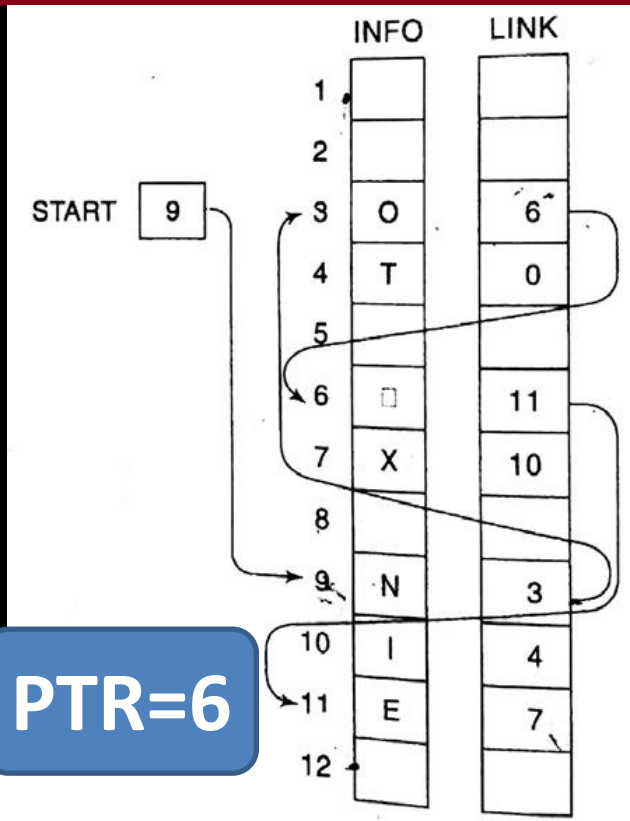


Fig. 5.8 *PTR : = LINK[PTR]*



PTR=6

1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR now points to the next node.]
   [End of Step 2 loop.]
5. Exit.

While Executing

CSE 203: Data Structure

Data Structure

Traversing elements of a Link List

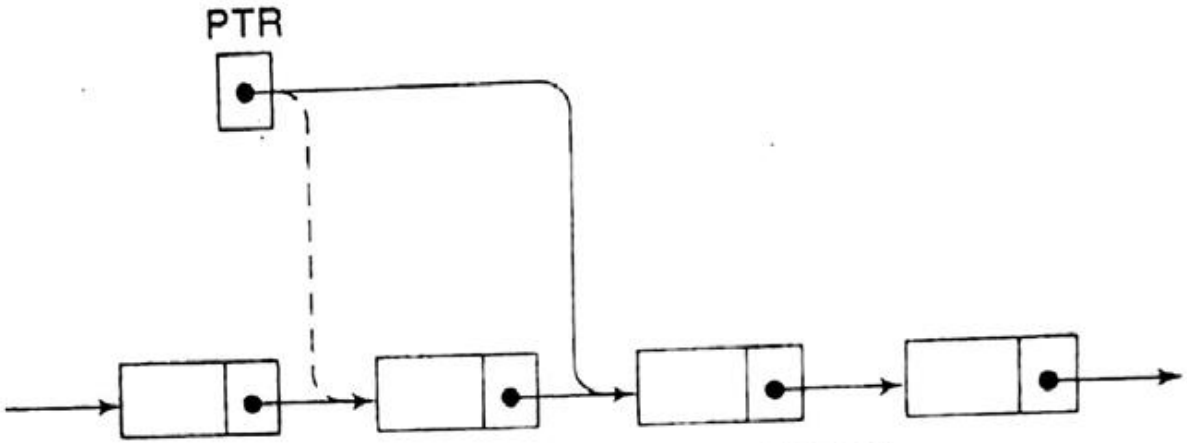
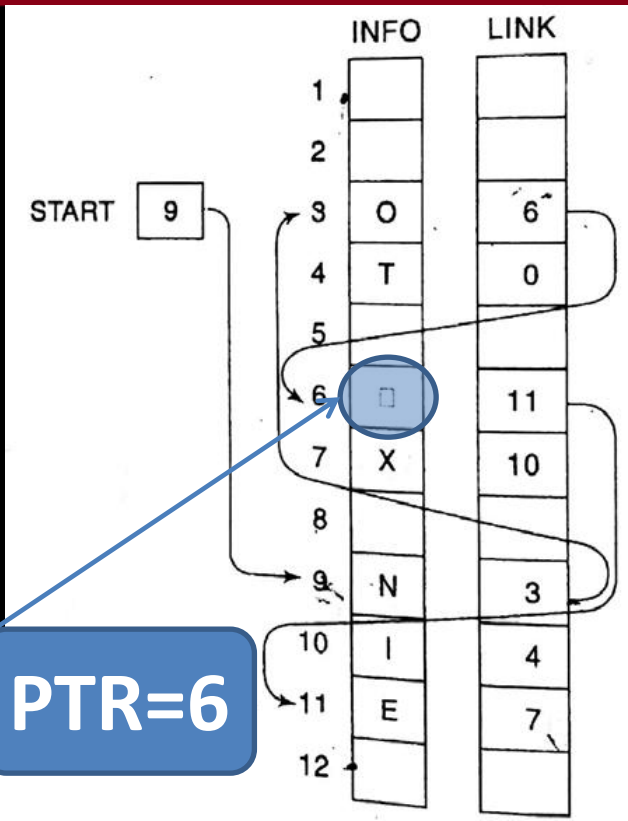


Fig. 5.8 *PTR : = LINK[PTR]*



PTR=6

1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR now points to the next node.]
   [End of Step 2 loop.]
5. Exit.

While Executing

CSE 203: Data Structure

Data Structure

Traversing elements of a Link List

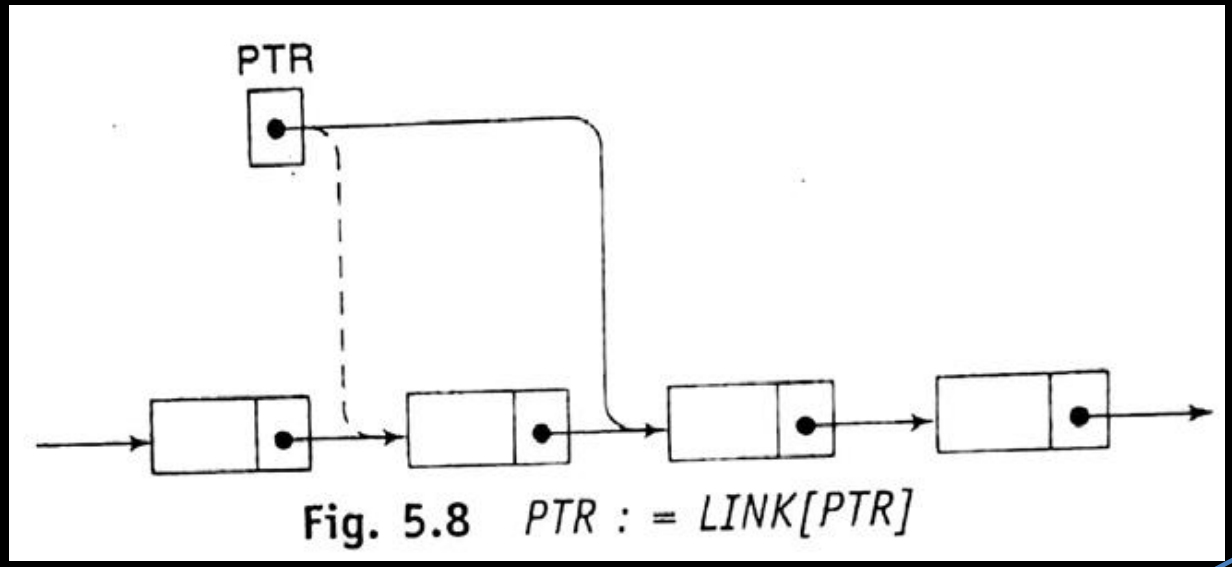
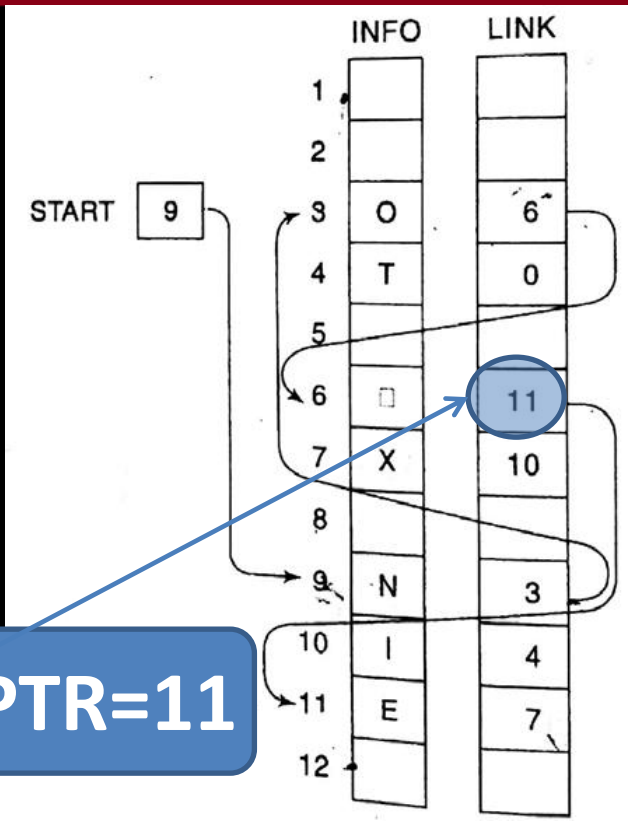


Fig. 5.8 *PTR : = LINK[PTR]*

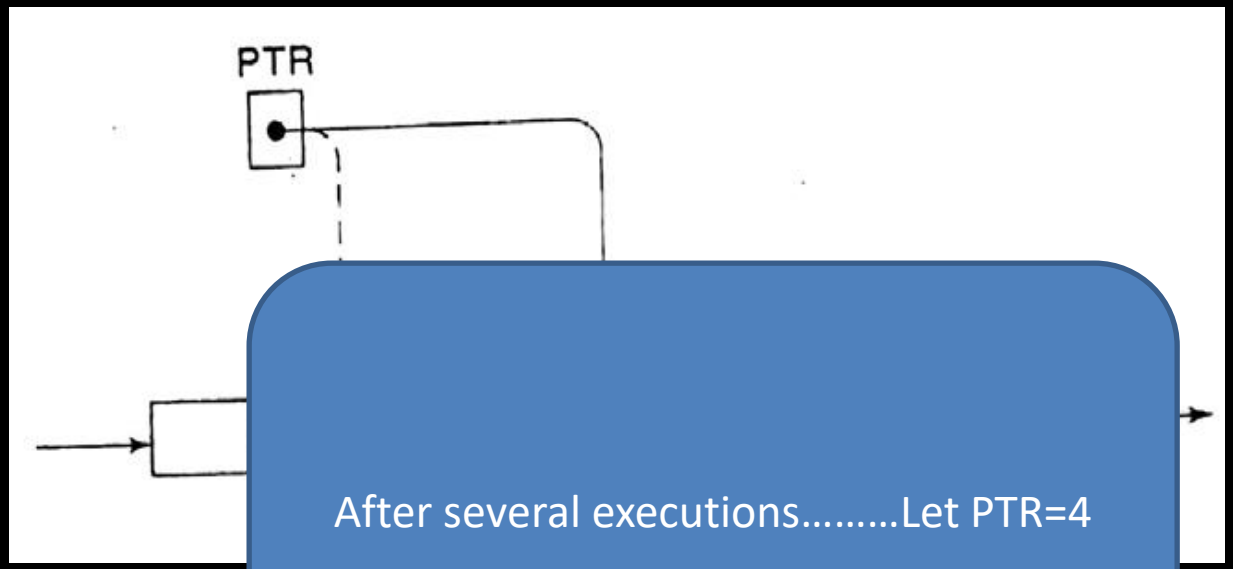


PTR=11

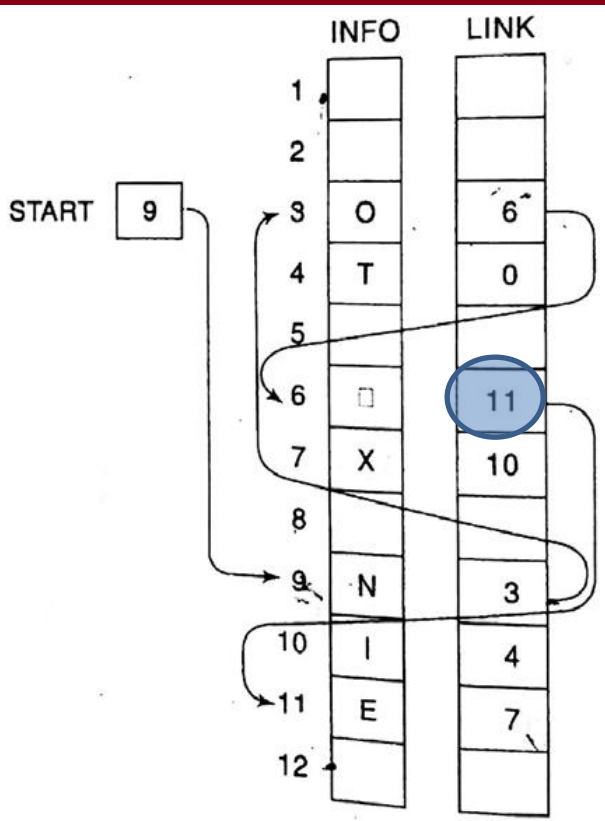
1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR now points to the next node.]
   [End of Step 2 loop.]
5. Exit.

While Executing

Traversing elements of a Link List



After several executions……Let PTR=4



1. Set P .]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR] ts to the next node.]

[End of Step 2 loop.]

5. Exit.

While Executing

CSE 203: Data Structure

Data Structure

Traversing elements of a Link List

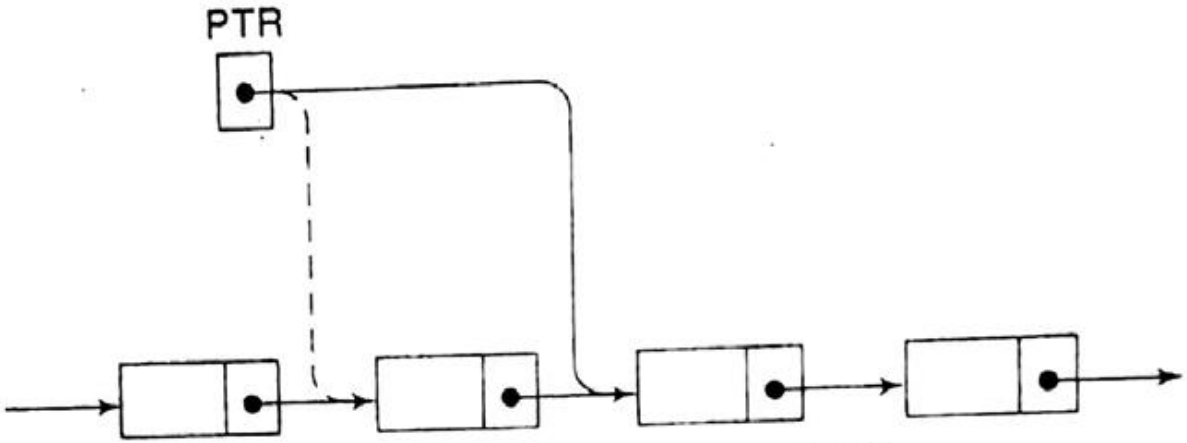
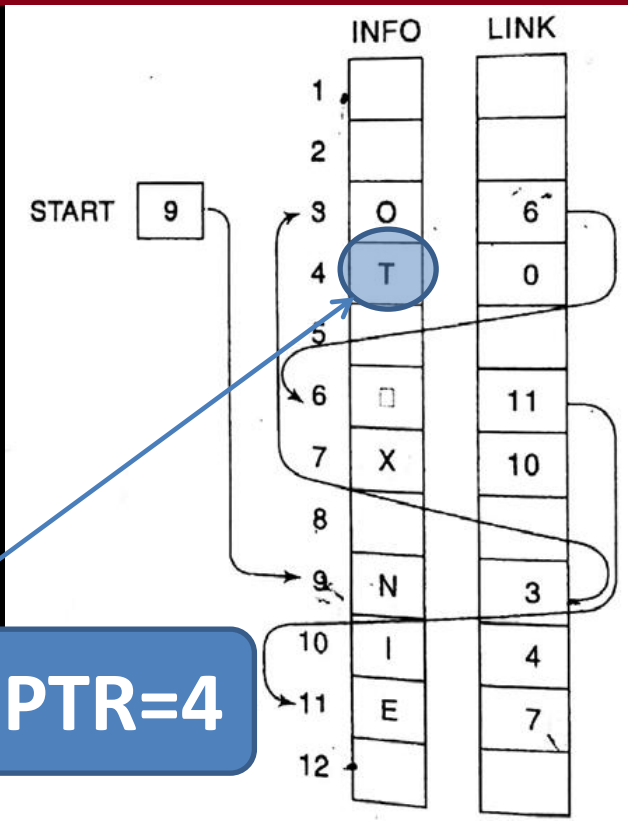


Fig. 5.8 *PTR : = LINK[PTR]*



PTR=4

1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR]. While Executing
4. Set PTR := LINK[PTR]. [PTR now points to the next node.]
   [End of Step 2 loop.]
5. Exit.

CSE 203: Data Structure

Data Structure

Traversing elements of a Link List

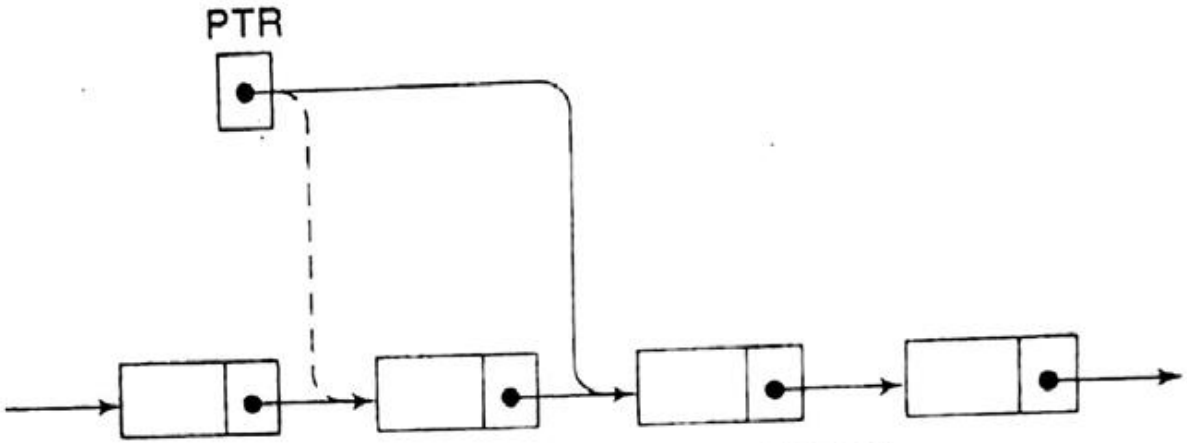
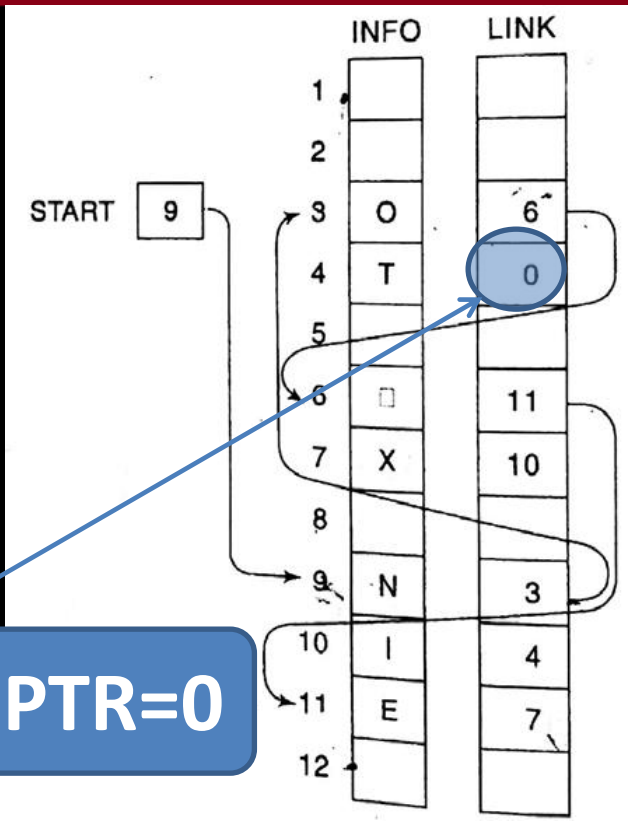


Fig. 5.8 *PTR : = LINK[PTR]*



PTR=0

1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR now points to the next node.]
   [End of Step 2 loop.]
5. Exit.

While Executing

CSE 203: Data Structure

Data Structure

Traversing elements of a Link List

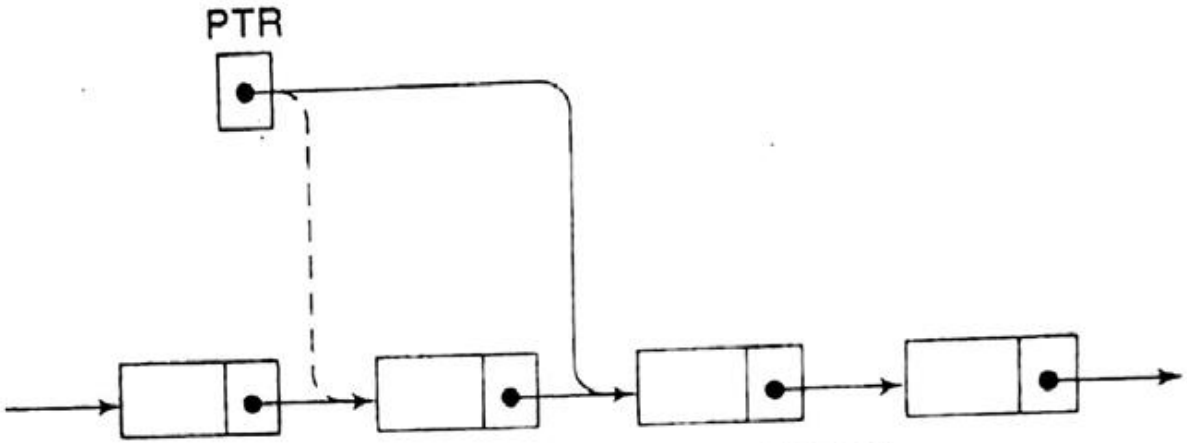
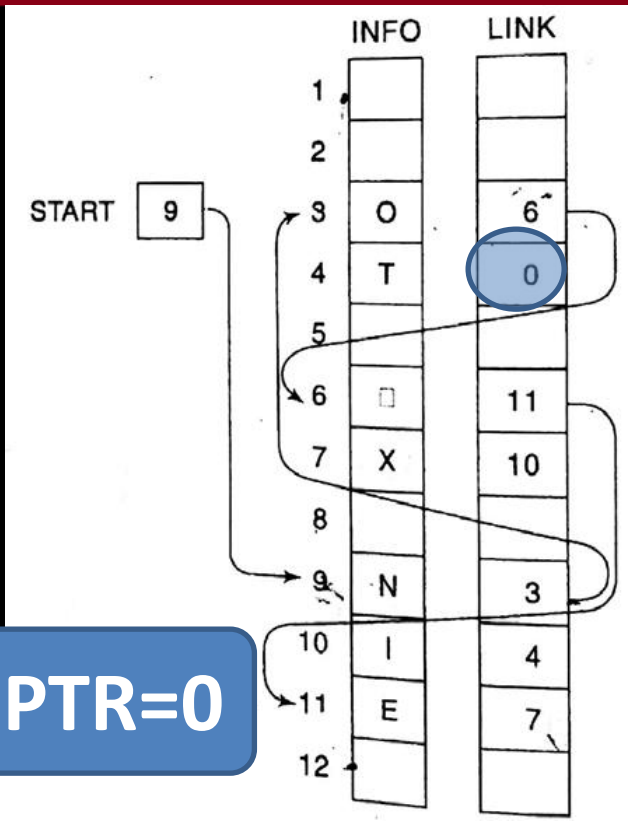


Fig. 5.8 *PTR : = LINK[PTR]*



PTR=0

1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR now points to the next node.]
   [End of Step 2 loop.]
5. Exit.

While Executing

CSE 203: Data Structure

Data Structure

Traversing elements of a Link List

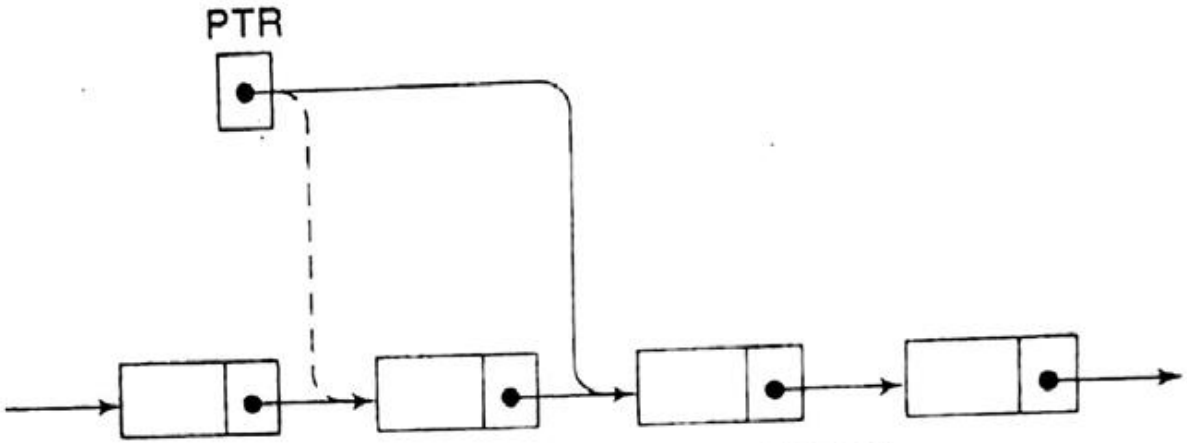
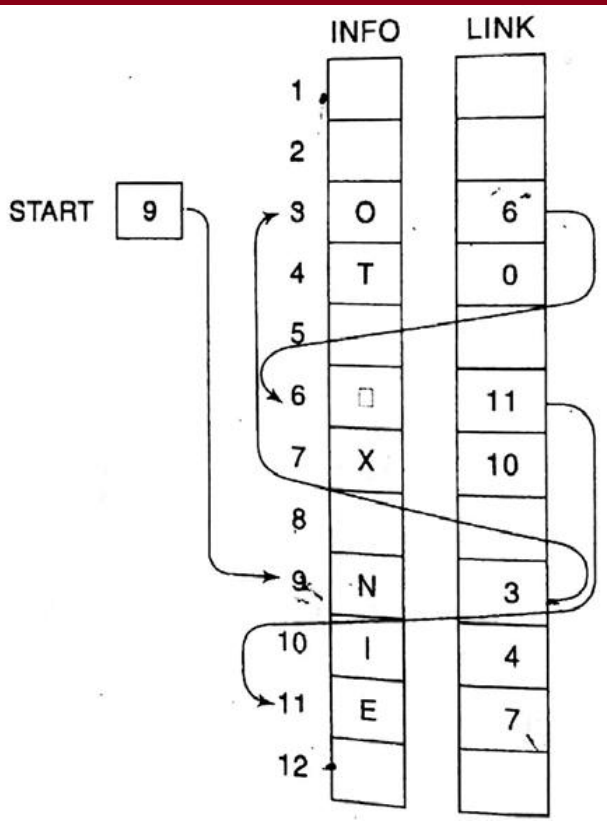


Fig. 5.8 *PTR : = LINK[PTR]*



1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR now points to the next node.]
   [End of Step 2 loop.]
5. Exit.

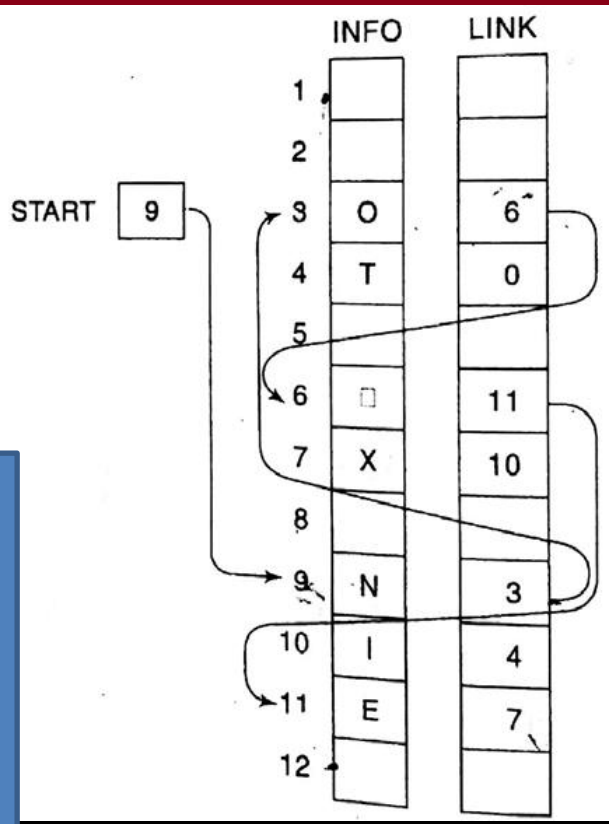
While Executing

CSE 203: Data Structure

Data Structure

Traversing elements of a Link List

1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR now points to the next n
   [End of Step 2 loop.]
5. Exit.



Code window

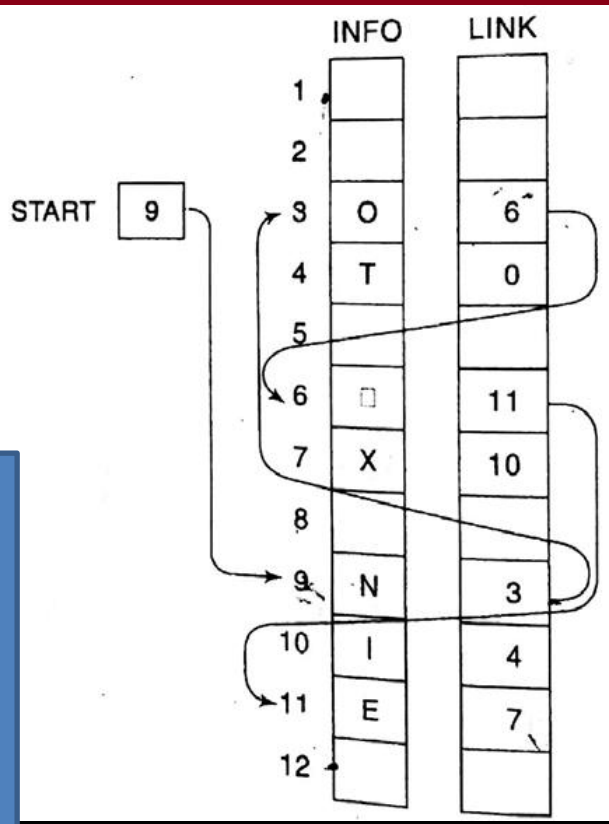
Implementing in C

CSE 203: Data Structure

Data Structure

Traversing elements of a Link List

1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR now points to the next n
   [End of Step 2 loop.]
5. Exit.



Identify the variables, their types and the necessary data structures.

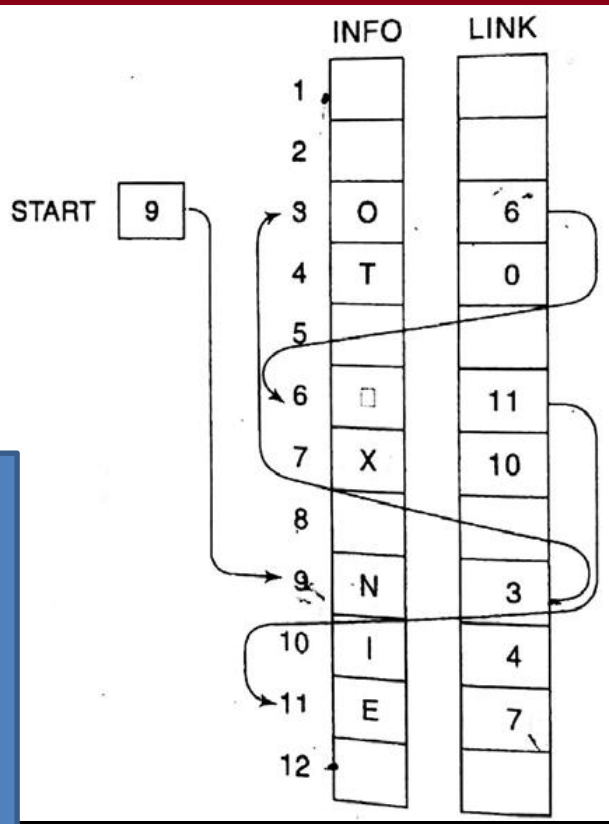
Implementing in C

CSE 203: Data Structure

Data Structure

Traversing elements of a Link List

1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR now points to the next n
   [End of Step 2 loop.]
5. Exit.



Identify the variables, their types and the necessary data structures.

Here,

There are four variables.

Three of integer type and one of character type.

Two of single valued and two of array types.

Now declare them accordingly.

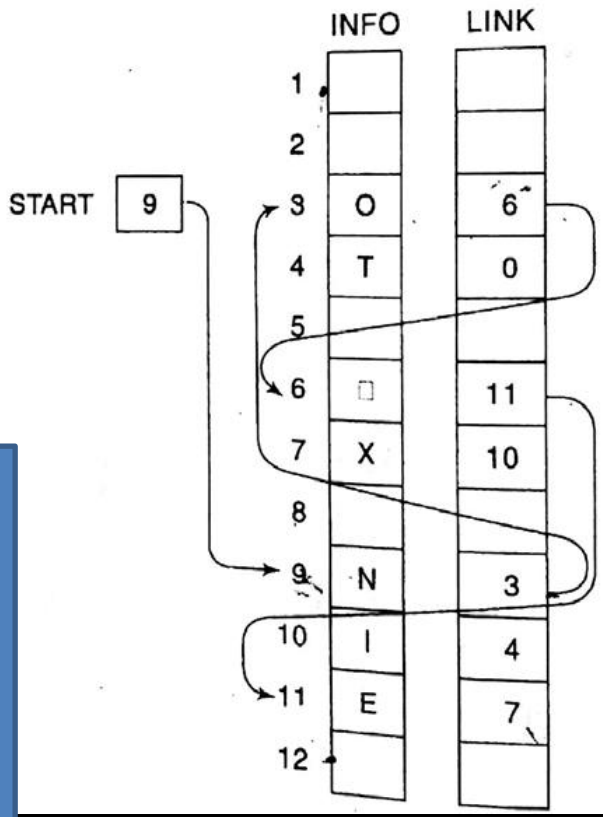
Implementing in C

CSE 203: Data Structure

Data Structure

Traversing elements of a Link List

1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR now points to the next n
   [End of Step 2 loop.]
5. Exit.




```
void main()
{  int START, PTR, LINK[12]={-2, -2, 6, -1, -2,  11, 10, -2, 3, 4, 7, -2};
char INFO[12]={“  OT  X NIE “};
```

Two space

Single space

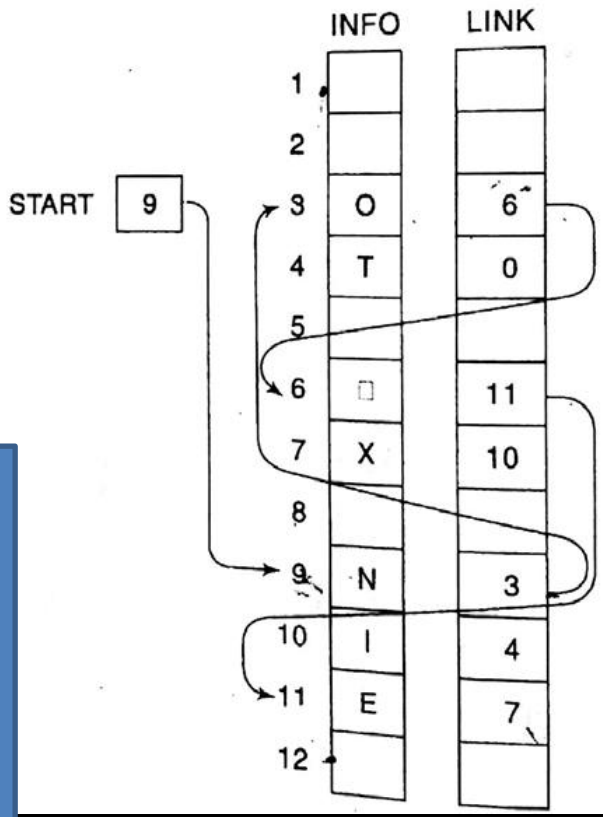
Implementing in C

CSE 203: Data Structure

Data Structure

Traversing elements of a Link List

1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR now points to the next n
   [End of Step 2 loop.]
5. Exit.



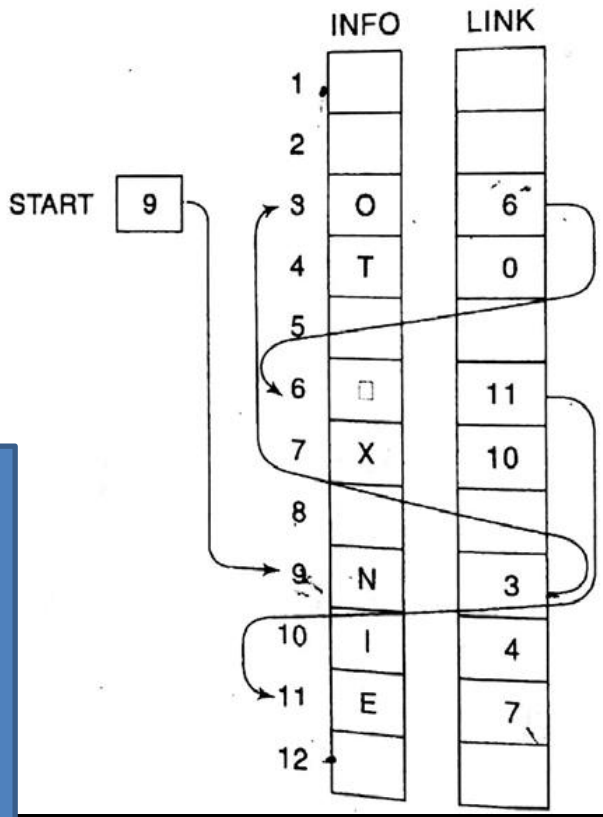

```
void main()
{  int START, PTR, LINK[12]={-2, -2, 6, -1, -2,  11, 10, -2, 3, 4, 7, -2};
char INFO[12]={"  OT  X NIE "};
PTR=START;
```

CSE 203: Data Structure

Data Structure

Traversing elements of a Link List

1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR now points to the next n
   [End of Step 2 loop.]
5. Exit.



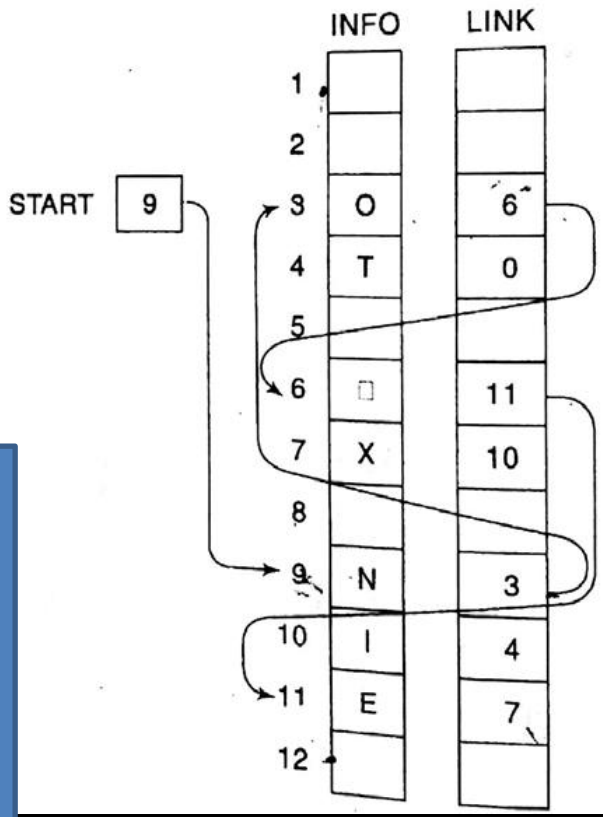

```
void main()
{  int START, PTR, LINK[12]={-2, -2, 6, -1, -2,  11, 10, -2, 3, 4, 7, -2};
char INFO[12]={"  OT  X NIE "};
PTR=START;
while(PTR!=-1)
{
```

CSE 203: Data Structure

Data Structure

Traversing elements of a Link List

1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR now points to the next n
   [End of Step 2 loop.]
5. Exit.




```
void main()
{  int START, PTR, LINK[12]={-2, -2, 6, -1, -2,  11, 10, -2, 3, 4, 7, -2};
char INFO[12]={“  OT  X NIE “};
PTR=START;
while(PTR!=-1)
{
printf(“%d, ”, INFO[PTR]);
```

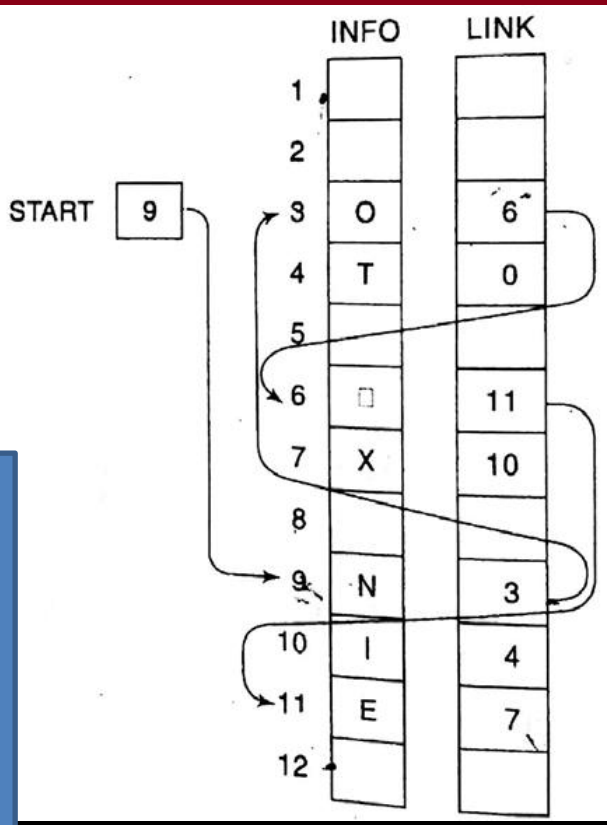
CSE 203: Data Structure

Data Structure

Traversing elements of a Link List

1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR now points to the next n
   [End of Step 2 loop.]
5. Exit.

```
void main()
{  int START, PTR, LINK[12]={-2, -2, 6, -1, -2,  11, 10, -2, 3, 4, 7, -2};
char INFO[12]={“  OT  X NIE “};
PTR=START;
while(PTR!=-1)
{
printf(“%d, ”, INFO[PTR]);
PTR=LINK[PTR];
}
}
```



Additional operations:

- Counting elements
- Printing elements

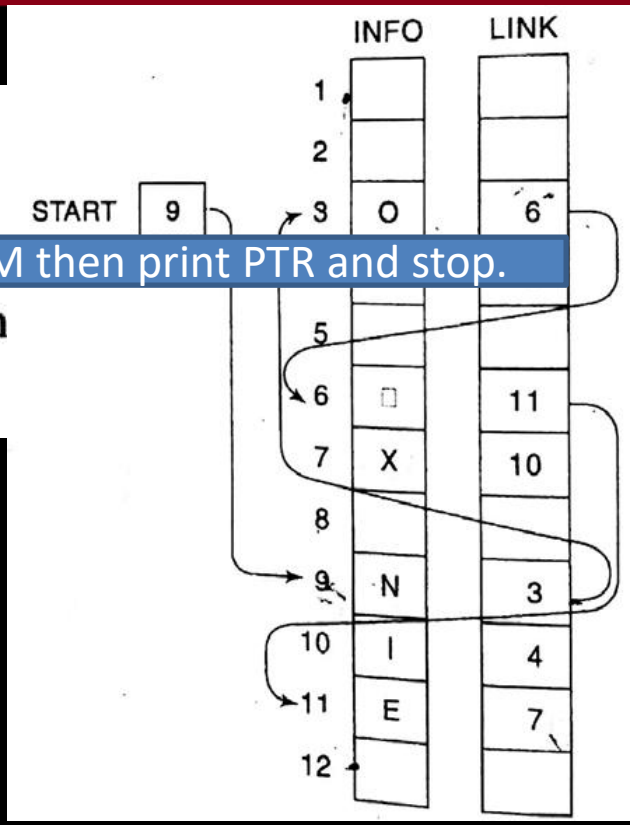
Searching an element in a Link List

What to do?

Searching an element in a Link List

1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR now points to the next n
   [End of Step 2 loop.]
5. Exit.

If INFO[PTR]==ITEM then print PTR and stop.

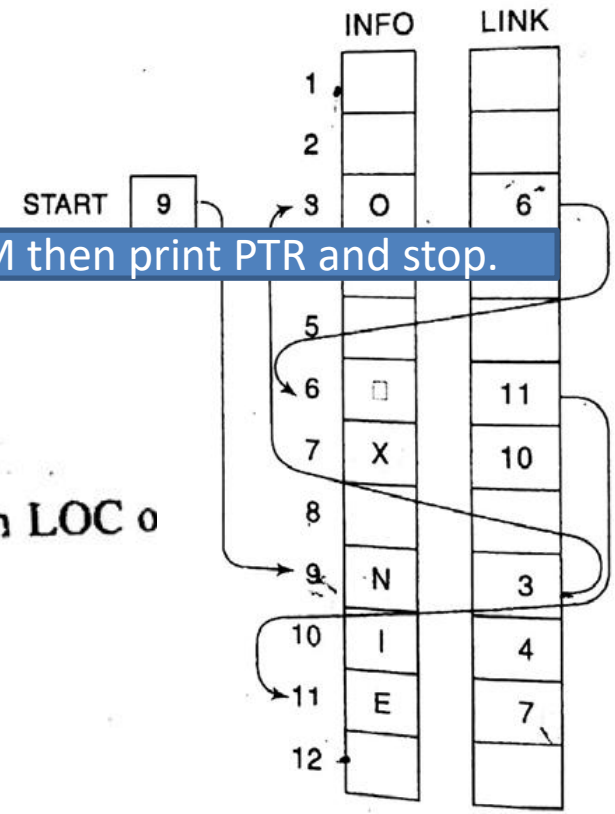


Searching an element in a Link List

1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR ... the next n
   [End of Step 2 loop.]
5. Exit.

If INFO[PTR]==ITEM then print PTR and stop.

For searching



SEARCH(INFO, LINK, START, ITEM, LOC)
LIST is a linked list in memory. This algorithm finds the location LOC o
node where ITEM first appears in LIST, or sets LOC = NULL.

1. Set PTR := START.
2. Repeat Step 3 while PTR ≠ NULL:
3. If ITEM = INFO[PTR], then:
   Set LOC := PTR, and Exit.
   Else:
   Set PTR := LINK[PTR]. [PTR now points to the next node.]
   [End of If structure.]
   [End of Step 2 loop.]
4. [Search is unsuccessful.] Set LOC := NULL.
5. Exit.

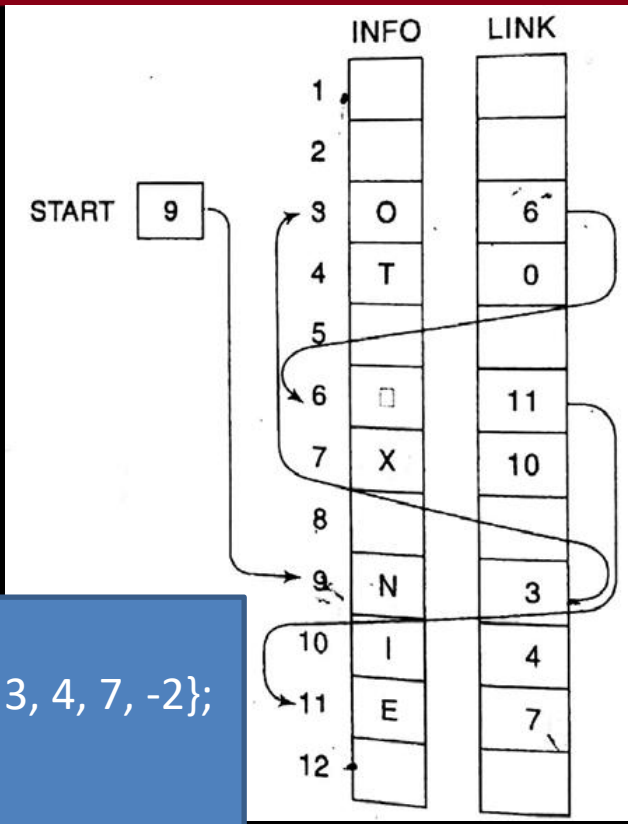
CSE 203: Data Structure

Data Structure

Searching an element in a Link List

SEARCH(INFO, LINK, START, ITEM, LOC)
LIST is a linked list in memory. This algorithm finds the location LOC o
node where ITEM first appears in LIST, or sets LOC = NULL.

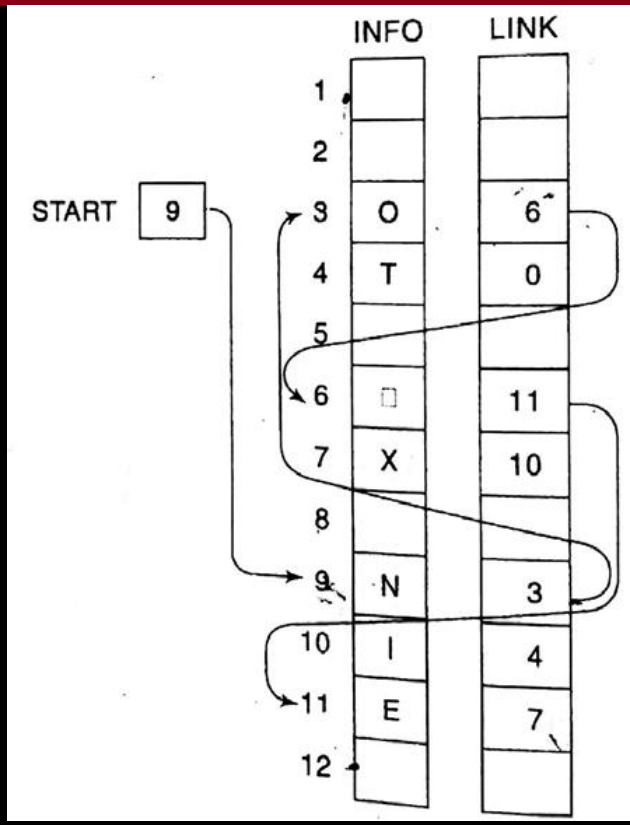
1. Set PTR := START.
2. Repeat Step 3 while PTR ≠ NULL:
3. If ITEM = INFO[PTR], then:
   Set LOC := PTR, and Exit.
   Else:
   Set PTR := LINK[PTR]. [PTR now points to the next node.]
   [End of If structure.]
   [End of Step 2 loop.]
4. [Search is unsuccessful.] Set LOC := NULL.
5. Exit




```
void main()
{  int START, PTR, LOC, LINK[12]={-2, -2, 6, -1, -2,  11, 10, -2, 3, 4, 7, -2};
char INFO[12]={"  OT  X NIE "}, ITEM='X';
PTR=START;
while(PTR!=-1)
{ if (ITEM==INFO[PTR])
            {LOC=PTR; printf("%d, ",LOC); return;}
  else PTR=LINK[PTR];
} printf("Item is not found in the array");
}
```

Deleting an element from the Link List

1. Deleting first element
2. Deleting last element
3. Deleting a given ITEM



1. Deleting First element

Set START=LINK[START]

2. Deleting Last element

Set PTR=START

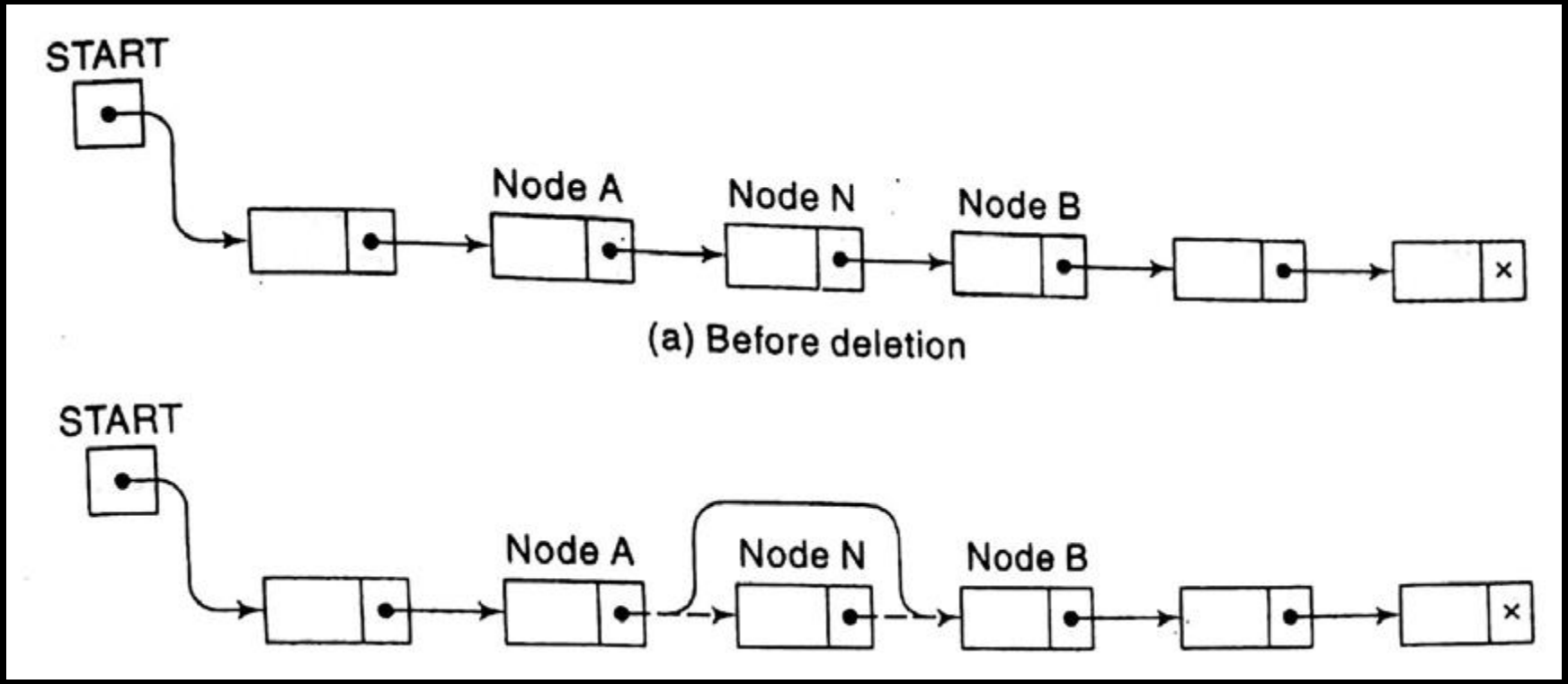
Repeat while LINK[LINK[PTR]]≠NULL

PTR=LINK[PTR]

Set LINK[PTR]=NULL

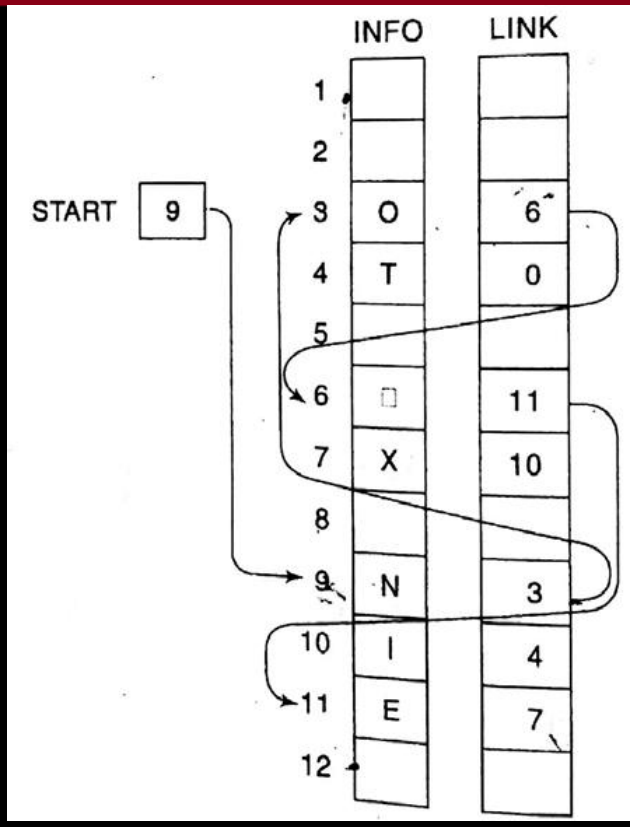
Deleting an element from the Link List

3. Given an ITEM to delete



Deleting an element from the Link List

1. Set PTR=START and PrevPTR=START
2. If ITEM=INFO[PTR] then:
   Set START=LINK[PTR], and Exit
3. Repeat Step 4 an 5 while PTR≠NULL
4. PTR=LINK[PTR]
5. If ITEM=INFO[PTR] then:
   Set LINK[PrevPTR]=LINK[PTR]
   Else PrevPTR=PTR
6. Exit



3. Given an ITEM

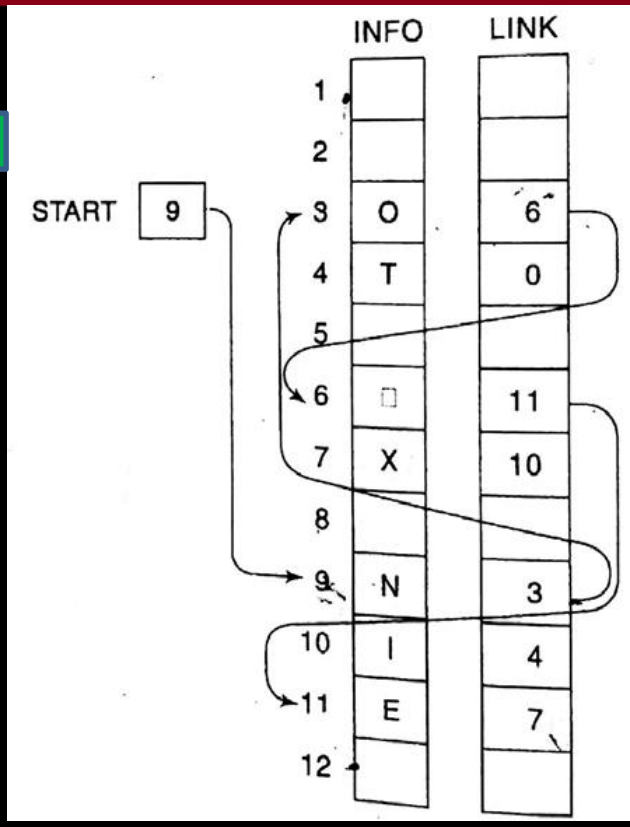
CSE 203: Data Structure

Data Structure

Deleting an element from the Link List

1. Set PTR=START and PrevPTR=START
2. If ITEM=INFO[PTR] then:
   Set START=LINK[PTR], and Exit
3. Repeat Step 4 an 5 while PTR≠NULL
4. PTR=LINK[PTR]
5. If ITEM=INFO[PTR] then:
   Set LINK[PrevPTR]=LINK[PTR]
   Else PrevPTR=PTR
6. Exit

While Executing



ITEM='E'

PTR=9

PrevPTR=9

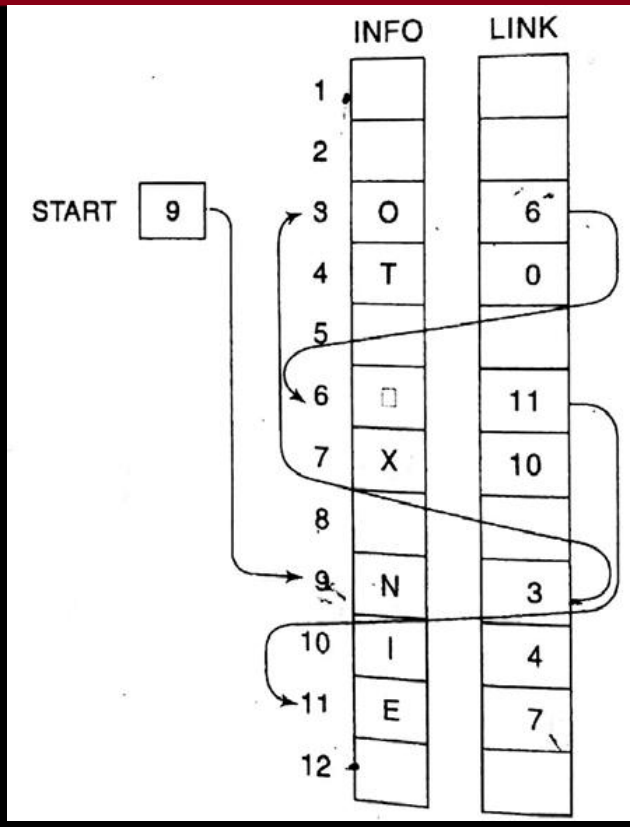
CSE 203: Data Structure

Data Structure

Deleting an element from the Link List

1. Set PTR=START and PrevPTR=START
2. If ITEM=INFO[PTR] then:
   Set START=LINK[PTR], and Exit
3. Repeat Step 4 an 5 while PTR≠NULL
4. PTR=LINK[PTR]
5. If ITEM=INFO[PTR] then:
   Set LINK[PrevPTR]=LINK[PTR]
   Else PrevPTR=PTR
6. Exit

While Executing



PTR=9
PrevPTR=9

ITEM='E'

INFO[9] is 'N'

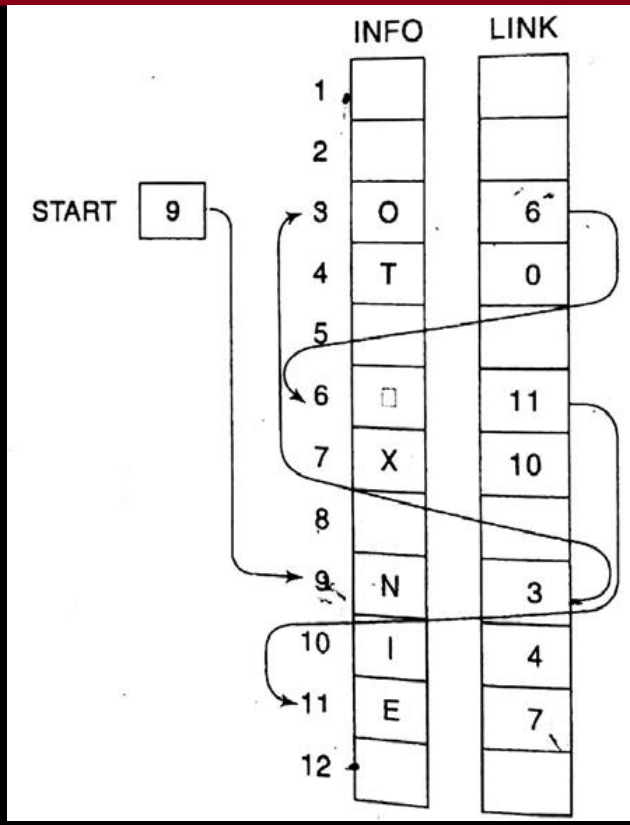
CSE 203: Data Structure

Data Structure

Deleting an element from the Link List

1. Set PTR=START and PrevPTR=START
2. If ITEM=INFO[PTR] then:
   Set START=LINK[PTR], and Exit
3. Repeat Step 4 an 5 while PTR≠NULL
4. PTR=LINK[PTR]
5. If ITEM=INFO[PTR] then:
   Set LINK[PrevPTR]=LINK[PTR]
   Else PrevPTR=PTR
6. Exit

While Executing



PTR=9
PrevPTR=9

ITEM='E'

INFO[9] is 'N'

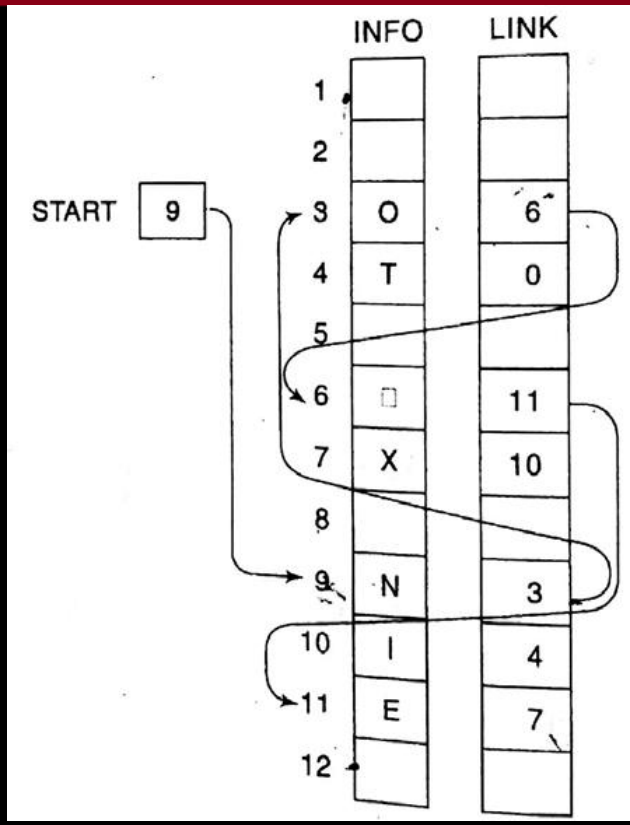
CSE 203: Data Structure

Data Structure

Deleting an element from the Link List

1. Set PTR=START and PrevPTR=START
2. If ITEM=INFO[PTR] then:
   Set START=LINK[PTR], and Exit
3. Repeat Step 4 an 5 while PTR≠NULL
4. PTR=LINK[PTR]
5. If ITEM=INFO[PTR] then:
   Set LINK[PrevPTR]=LINK[PTR]
   Else PrevPTR=PTR
6. Exit

While Executing



PTR=3 because LINK[PTR] is 3.
PrevPTR=9

ITEM='E'

INFO[9] is 'N'

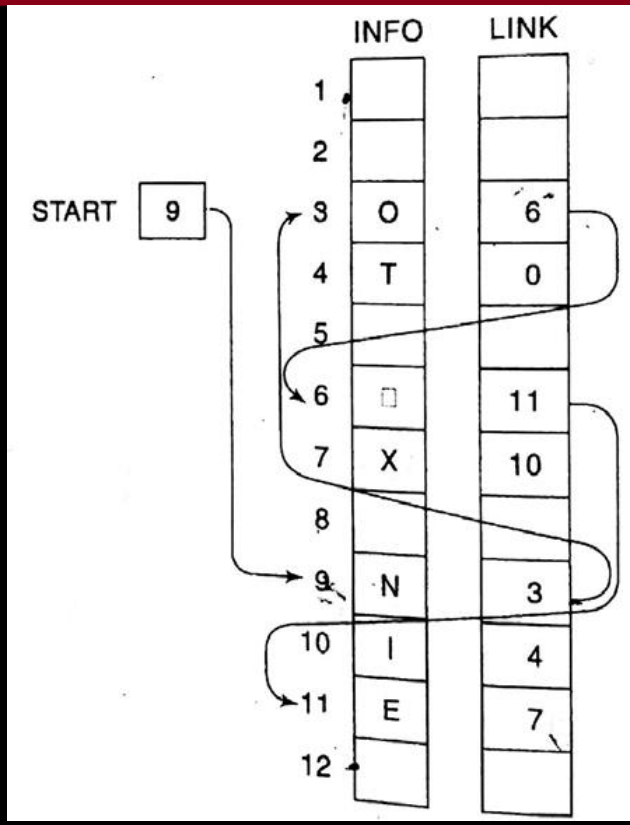
CSE 203: Data Structure

Data Structure

Deleting an element from the Link List

1. Set PTR=START and PrevPTR=START
2. If ITEM=INFO[PTR] then:
   Set START=LINK[PTR], and Exit
3. Repeat Step 4 an 5 while PTR≠NULL
4. PTR=LINK[PTR]
5. If ITEM=INFO[PTR] then:
   Set LINK[PrevPTR]=LINK[PTR]
   Else PrevPTR=PTR
6. Exit

While Executing



PTR=3 because LINK[PTR] is 3.
PrevPTR=9

ITEM='E'

INFO[3] is 'O'

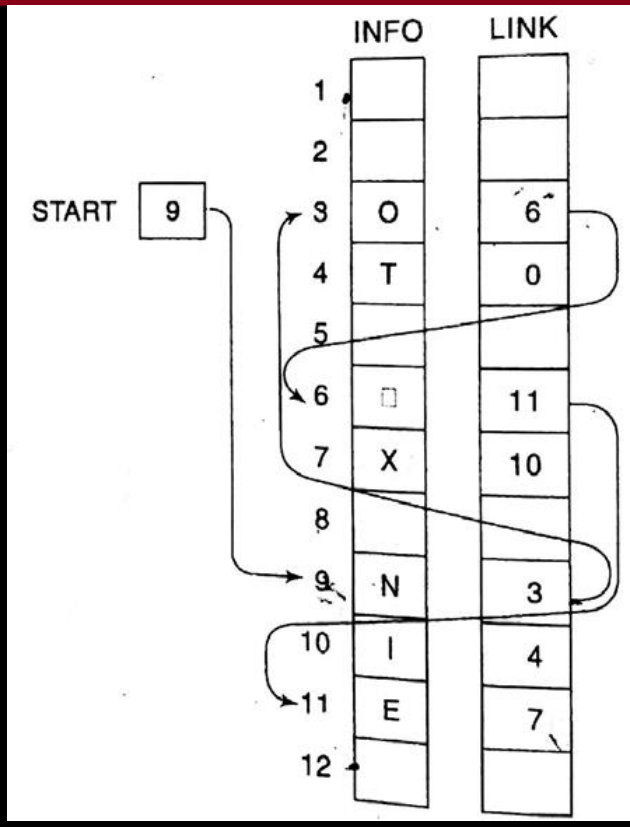
CSE 203: Data Structure

Data Structure

Deleting an element from the Link List

1. Set PTR=START and PrevPTR=START
2. If ITEM=INFO[PTR] then:
   Set START=LINK[PTR], and Exit
3. Repeat Step 4 an 5 while PTR≠NULL
4. PTR=LINK[PTR]
5. If ITEM=INFO[PTR] then:
   Set LINK[PrevPTR]=LINK[PTR]
   Else PrevPTR=PTR
6. Exit

While Executing



PTR=3 because LINK[PTR] is 3.
PrevPTR=3

ITEM='E'

INFO[3] is 'O'

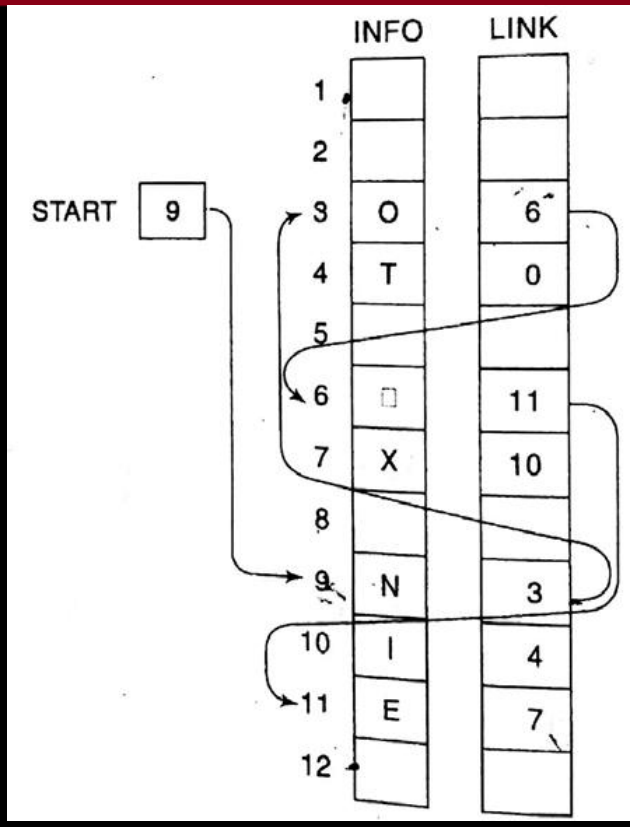
CSE 203: Data Structure

Data Structure

Deleting an element from the Link List

1. Set PTR=START and PrevPTR=START
2. If ITEM=INFO[PTR] then:
   Set START=LINK[PTR], and Exit
3. Repeat Step 4 an 5 while PTR≠NULL
4. PTR=LINK[PTR]
5. If ITEM=INFO[PTR] then:
   Set LINK[PrevPTR]=LINK[PTR]
   Else PrevPTR=PTR
6. Exit

While Executing



PTR=3 because LINK[PTR] is 3.
PrevPTR=3

ITEM='E'

INFO[3] is 'O'

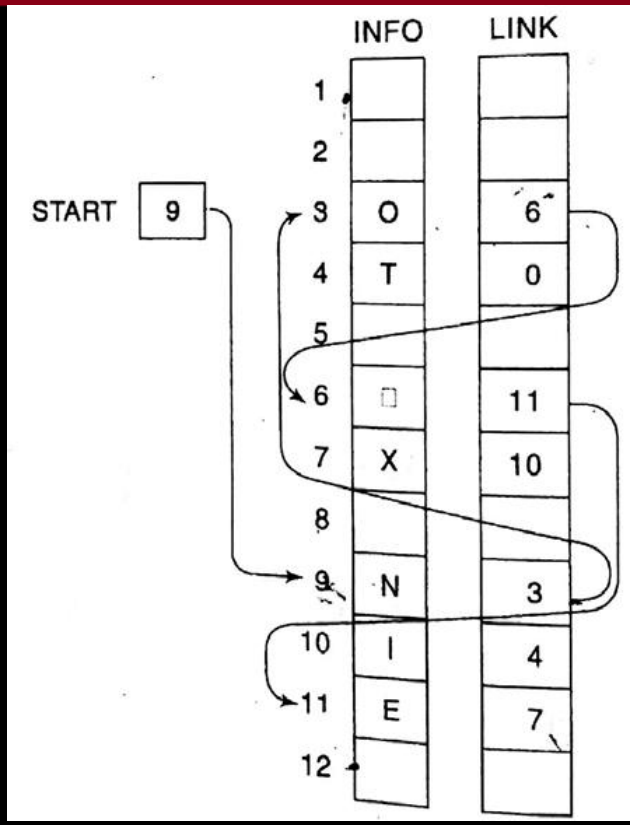
CSE 203: Data Structure

Data Structure

Deleting an element from the Link List

1. Set PTR=START and PrevPTR=START
2. If ITEM=INFO[PTR] then:
   Set START=LINK[PTR], and Exit
3. Repeat Step 4 an 5 while PTR≠NULL
4. PTR=LINK[PTR]
5. If ITEM=INFO[PTR] then:
   Set LINK[PrevPTR]=LINK[PTR]
   Else PrevPTR=PTR
6. Exit

While Executing



PTR=6 because LINK[3] is 6.
PrevPTR=3

ITEM='E'

INFO[3] is 'O'

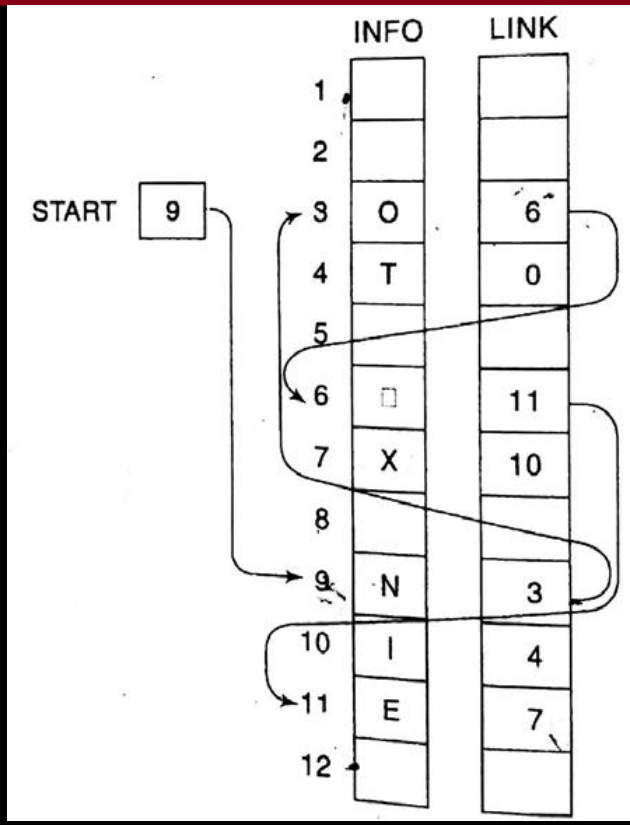
CSE 203: Data Structure

Data Structure

Deleting an element from the Link List

1. Set PTR=START and PrevPTR=START
2. If ITEM=INFO[PTR] then:
   Set START=LINK[PTR], and Exit
3. Repeat Step 4 an 5 while PTR≠NULL
4. PTR=LINK[PTR]
5. If ITEM=INFO[PTR] then:
   Set LINK[PrevPTR]=LINK[PTR]
   Else PrevPTR=PTR
6. Exit

While Executing



PTR=6 because LINK[3] is 6.
PrevPTR=3

ITEM='E'

INFO[6] is ' '

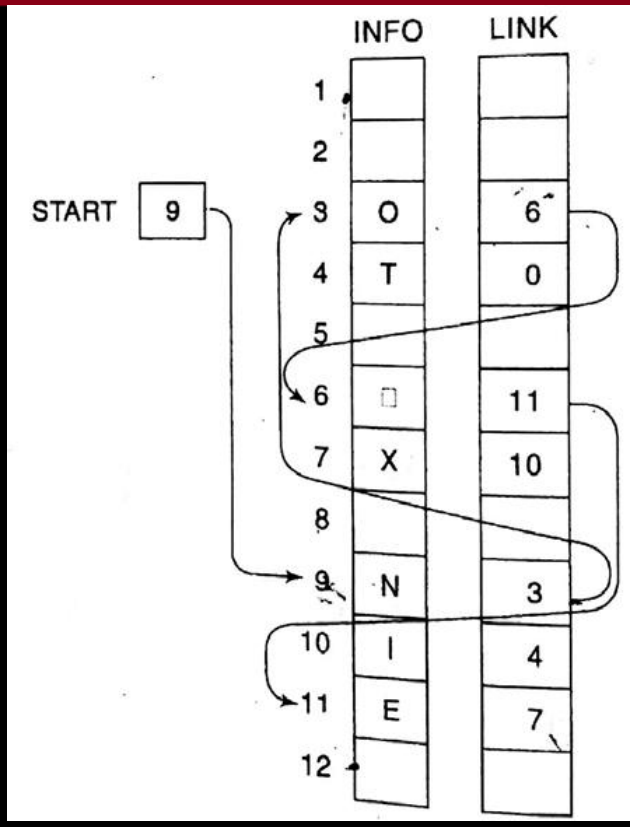
CSE 203: Data Structure

Data Structure

Deleting an element from the Link List

1. Set PTR=START and PrevPTR=START
2. If ITEM=INFO[PTR] then:
   Set START=LINK[PTR], and Exit
3. Repeat Step 4 an 5 while PTR≠NULL
4. PTR=LINK[PTR]
5. If ITEM=INFO[PTR] then:
   Set LINK[PrevPTR]=LINK[PTR]
   Else PrevPTR=PTR
6. Exit

While Executing



PTR=6 because LINK[3] is 6.
PrevPTR=6

ITEM='E'

INFO[6] is ' '

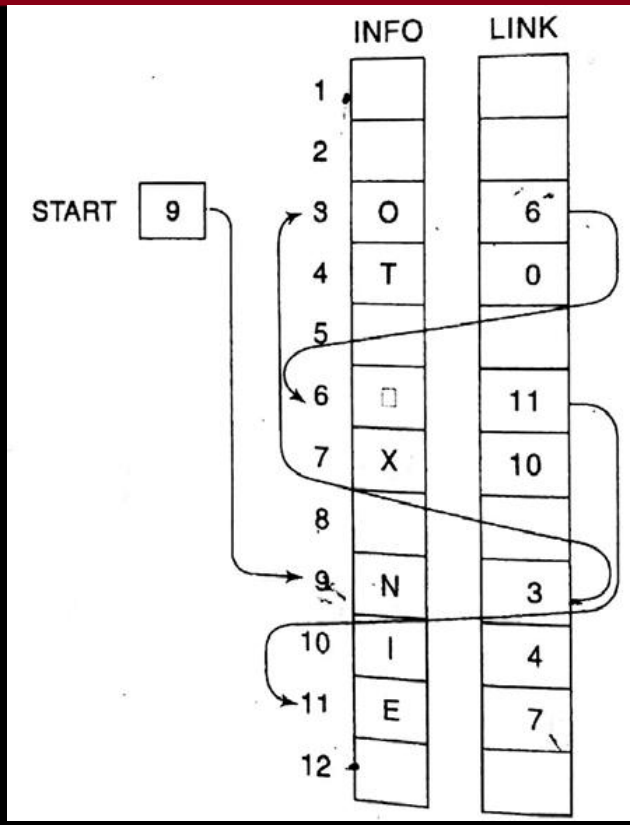
CSE 203: Data Structure

Data Structure

Deleting an element from the Link List

1. Set PTR=START and PrevPTR=START
2. If ITEM=INFO[PTR] then:
   Set START=LINK[PTR], and Exit
3. Repeat Step 4 an 5 while PTR≠NULL
4. PTR=LINK[PTR]
5. If ITEM=INFO[PTR] then:
   Set LINK[PrevPTR]=LINK[PTR]
   Else PrevPTR=PTR
6. Exit

While Executing



PTR=6 because LINK[3] is 6.
PrevPTR=6

ITEM='E'

INFO[6] is ' '

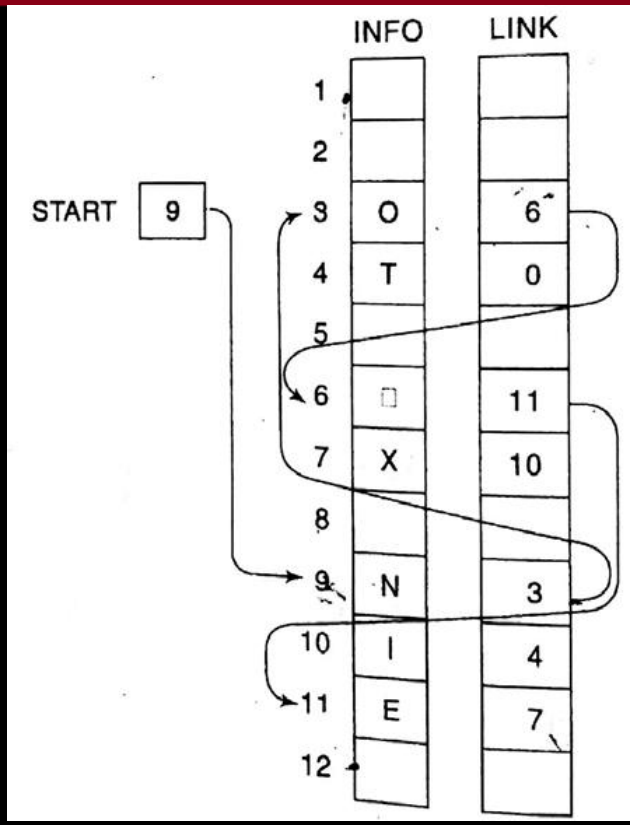
CSE 203: Data Structure

Data Structure

Deleting an element from the Link List

1. Set PTR=START and PrevPTR=START
2. If ITEM=INFO[PTR] then:
   Set START=LINK[PTR], and Exit
3. Repeat Step 4 an 5 while PTR≠NULL
4. PTR=LINK[PTR]
5. If ITEM=INFO[PTR] then:
   Set LINK[PrevPTR]=LINK[PTR]
   Else PrevPTR=PTR
6. Exit

While Executing



PTR=11 because LINK[6] is 11.
PrevPTR=6

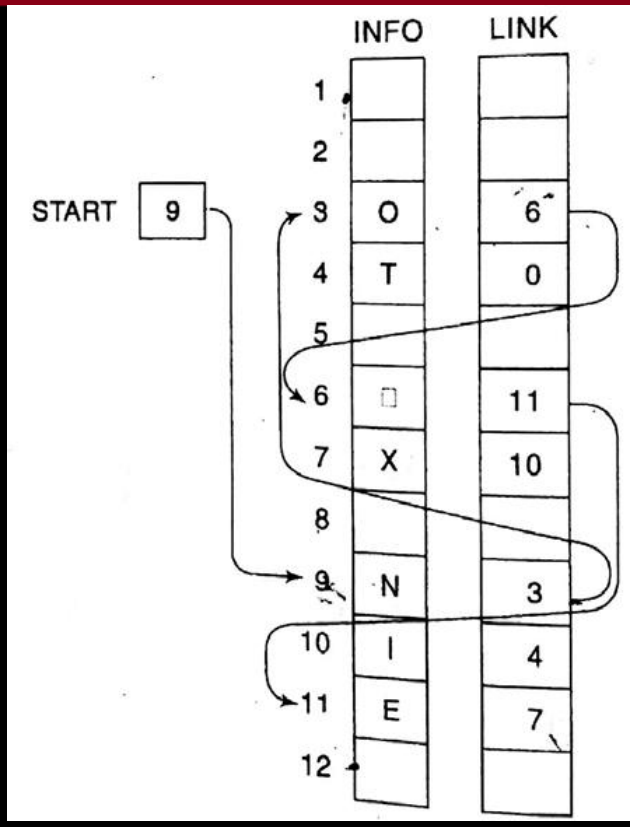
ITEM='E'

INFO[6] is ' '

Deleting an element from the Link List

1. Set PTR=START and PrevPTR=START
2. If ITEM=INFO[PTR] then:
   Set START=LINK[PTR], and Exit
3. Repeat Step 4 an 5 while PTR≠NULL
4. PTR=LINK[PTR]
5. If ITEM=INFO[PTR] then:
   Set LINK[PrevPTR]=LINK[PTR]
   Else PrevPTR=PTR
6. Exit

While Executing



ITEM='E'

PTR=11 because LINK[6] is 11.
PrevPTR=6

INFO[11] is 'E'

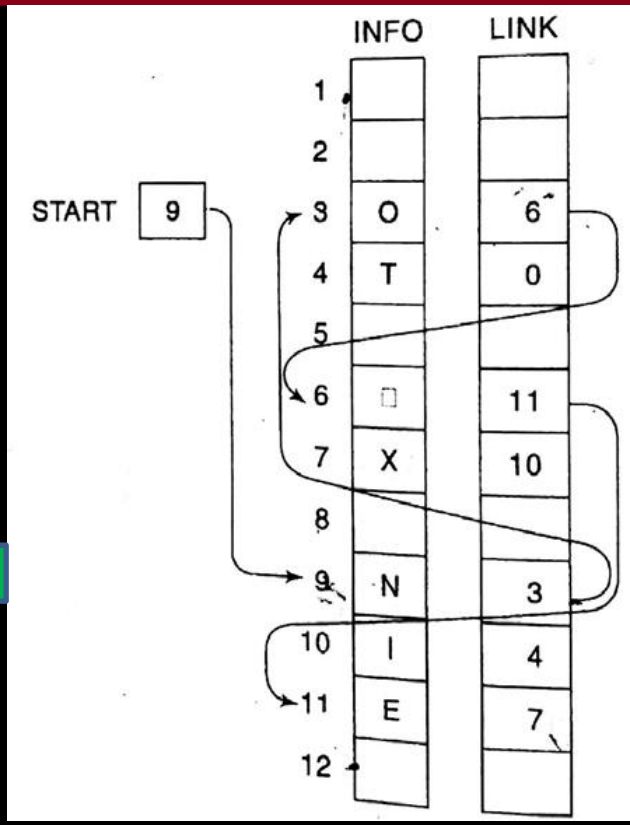
CSE 203: Data Structure

Data Structure

Deleting an element from the Link List

1. Set PTR=START and PrevPTR=START
2. If ITEM=INFO[PTR] then:
   Set START=LINK[PTR], and Exit
3. Repeat Step 4 an 5 while PTR≠NULL
4. PTR=LINK[PTR]
5. If ITEM=INFO[PTR] then:
   Set LINK[PrevPTR]=LINK[PTR], Exit
   Else PrevPTR=PTR
6. Exit

While Executing



ITEM='E'

INFO[11] is 'E'

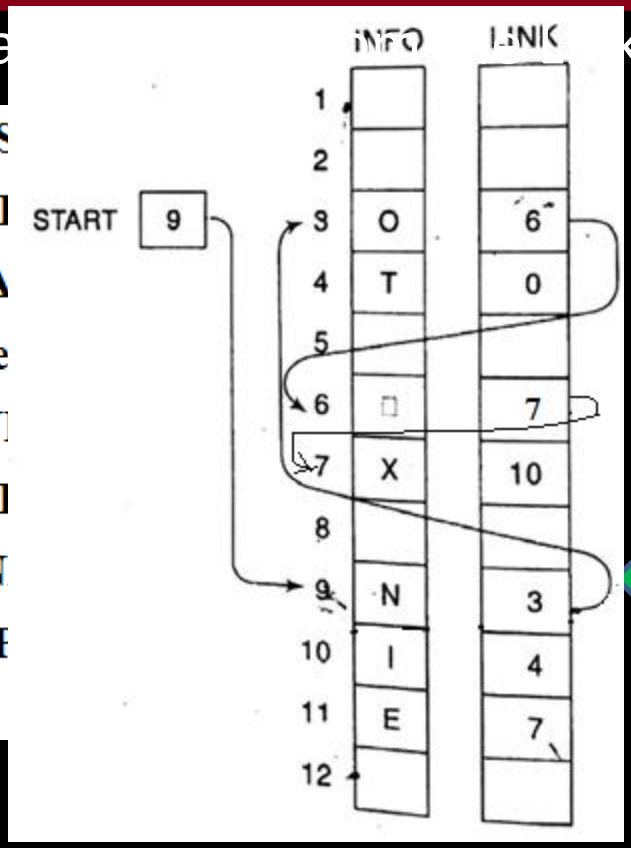
PTR=11 because LINK[6] is 11.
PrevPTR=6
LINK[6]=LINK[11], that is, LINK[6]=7

CSE 203: Data Structure

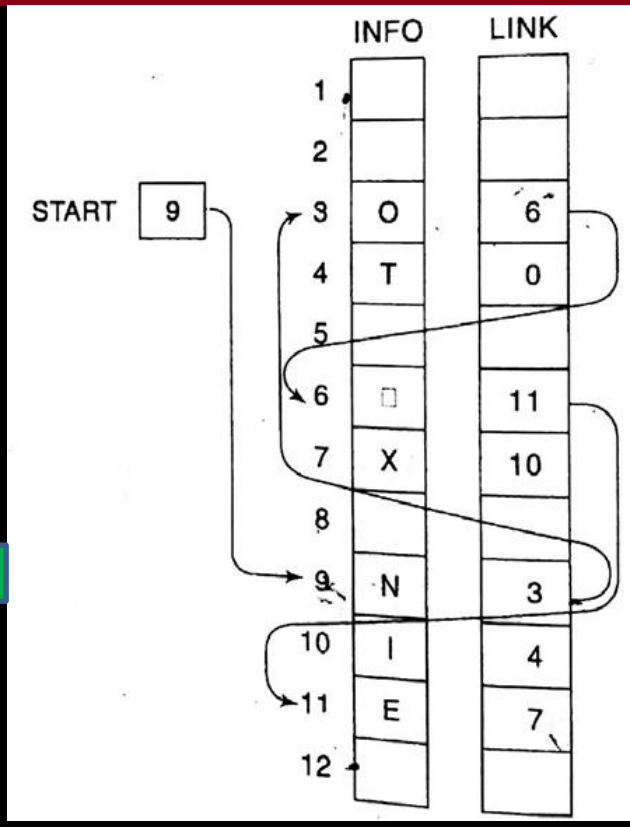
Data Structure

Deleting a … k List

1. Set PTR=S
2. If ITEM=I
   Set STA
3. Repeat Ste
4. PTR=LIN
5. If ITEM=I
   Set LIN
   Else PrevI
6. Exit



After Deletion



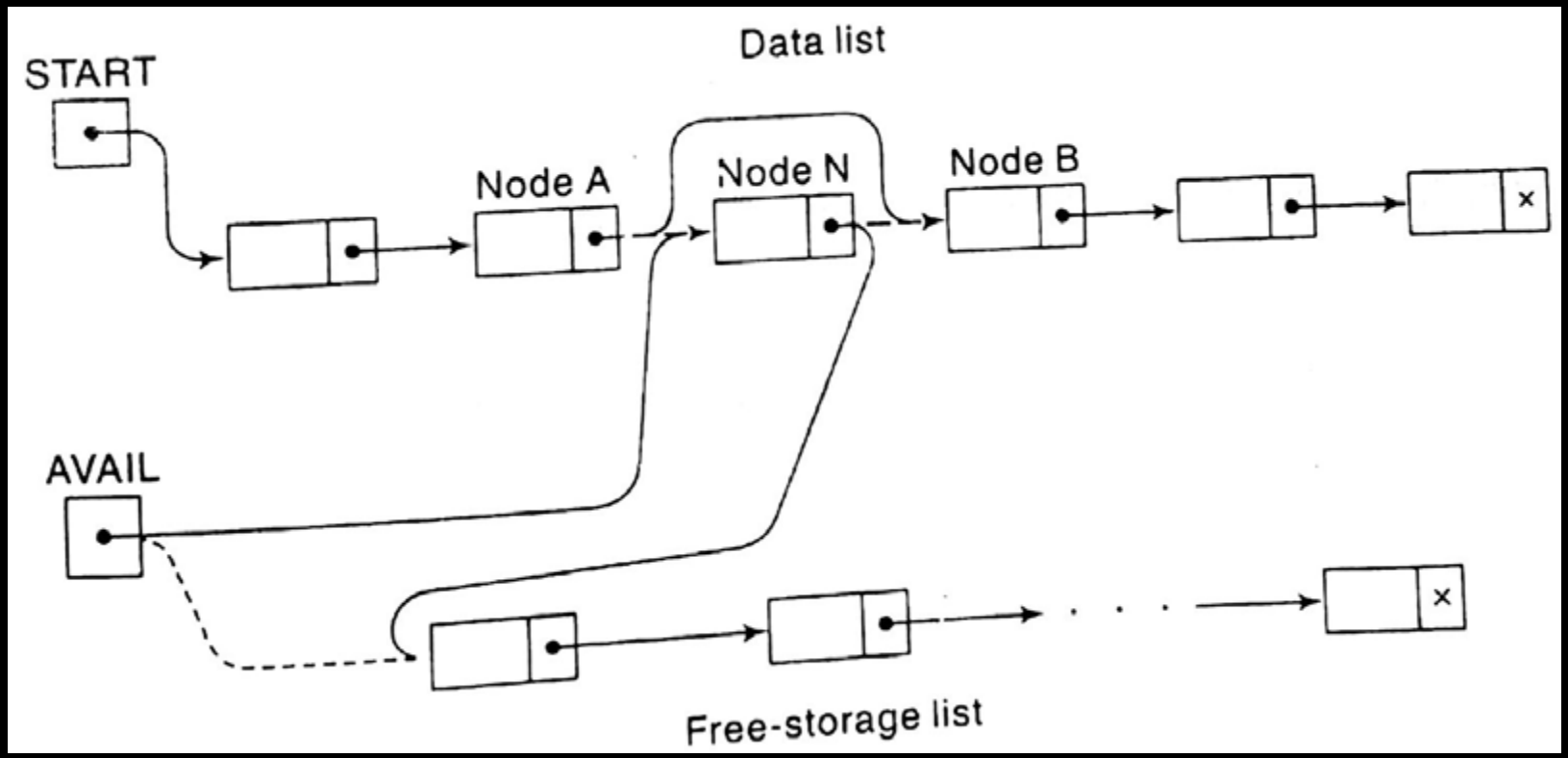
PTR=11 because LINK[6] is 11.
PrevPTR=6
LINK[6]=LINK[11], that is, LINK[6]=7

ITEM='E'

INFO[11] is 'E'

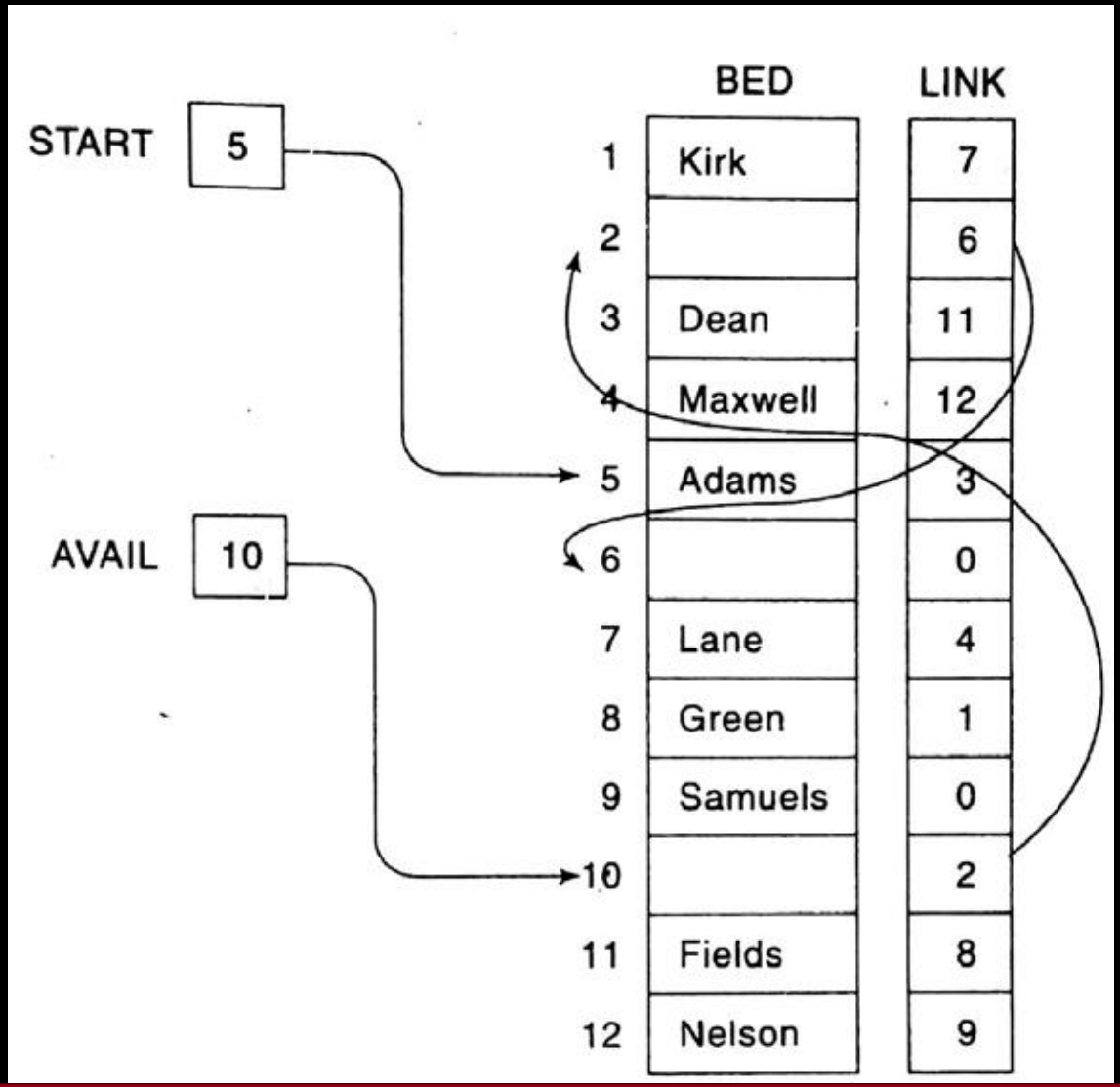
Deleting an element from the Link List

What to about AVAIL List?



Memory allocation and garbage collection of a Link List

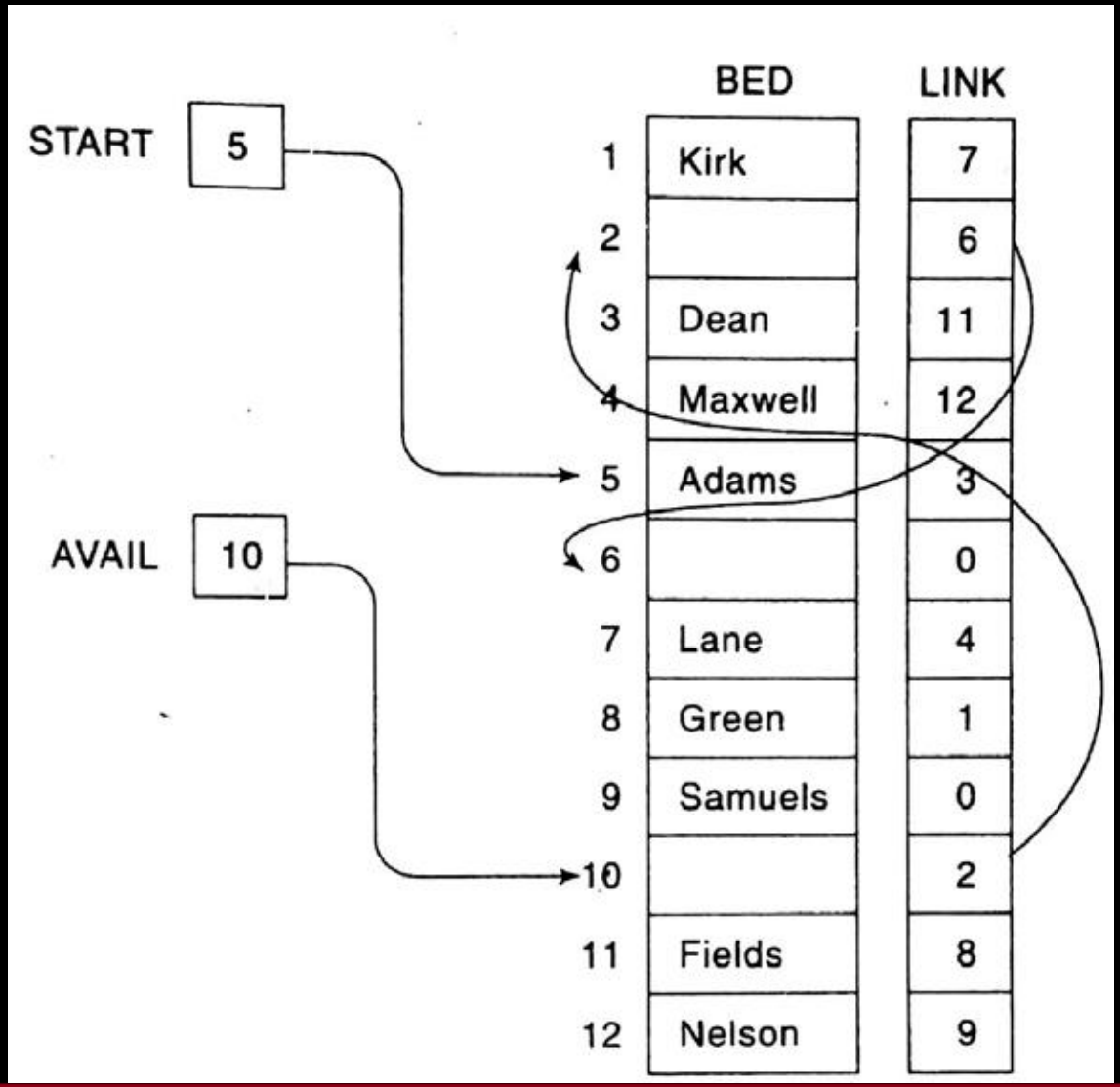
Memory allocation and garbage collection of a Link List



Memory allocation and garbage collection of a Link List

If an item is deleted from the list, how to collect the space for the purpose of future reuse?

What is OVERFLOW and UNDERFLOW?
When do these two operations happen?

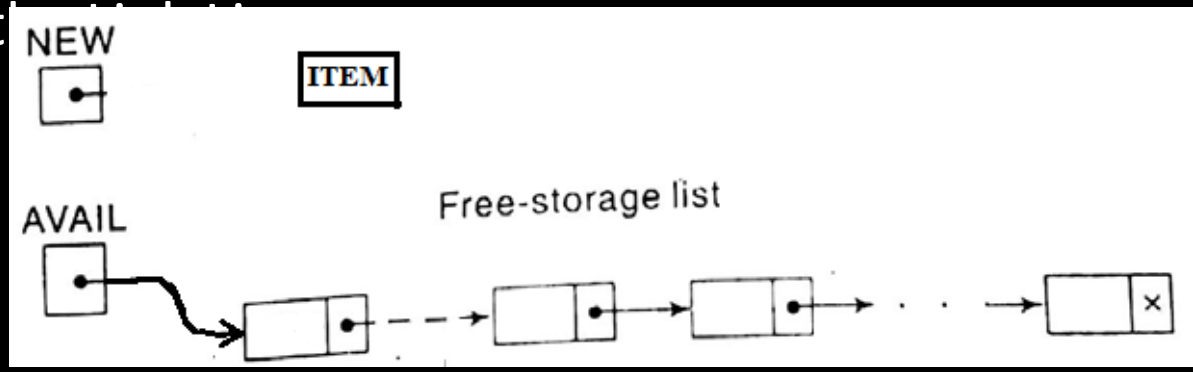


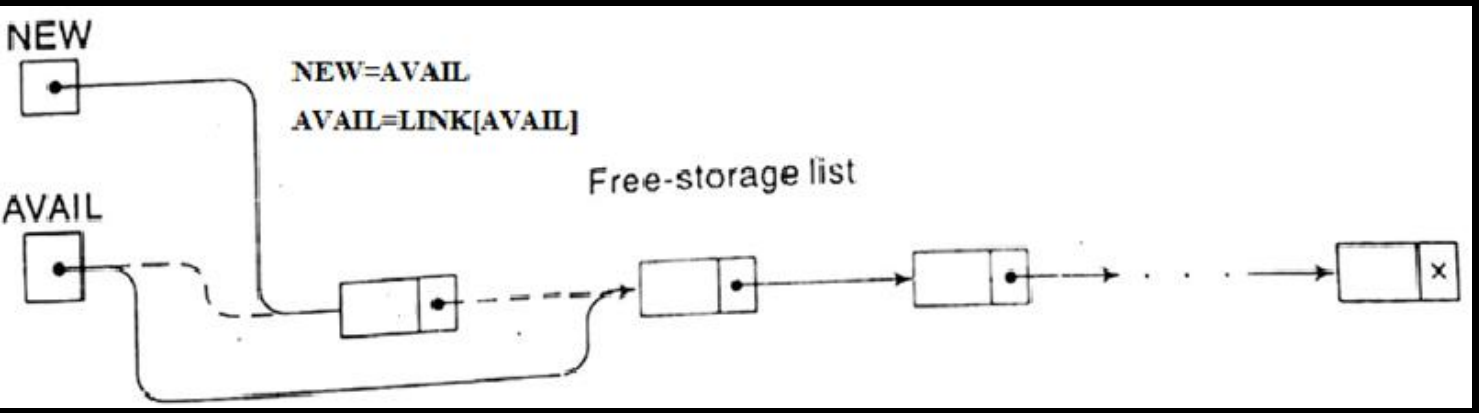
Inserting an element from the Link List

1. Inserting in an empty list
2. Inserting as a first element
3. Inserting at the last
4. Inserting ITEM at a given position

Inserting an element from t

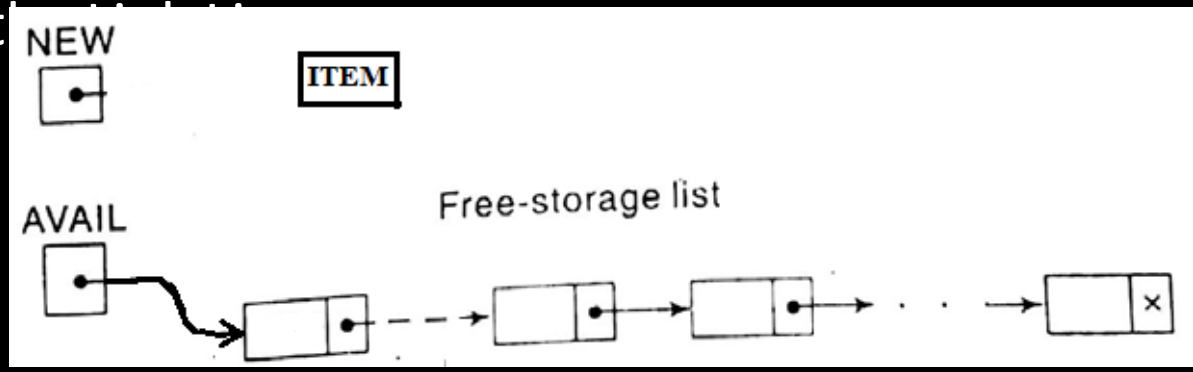
1. **Inserting in an empty list**

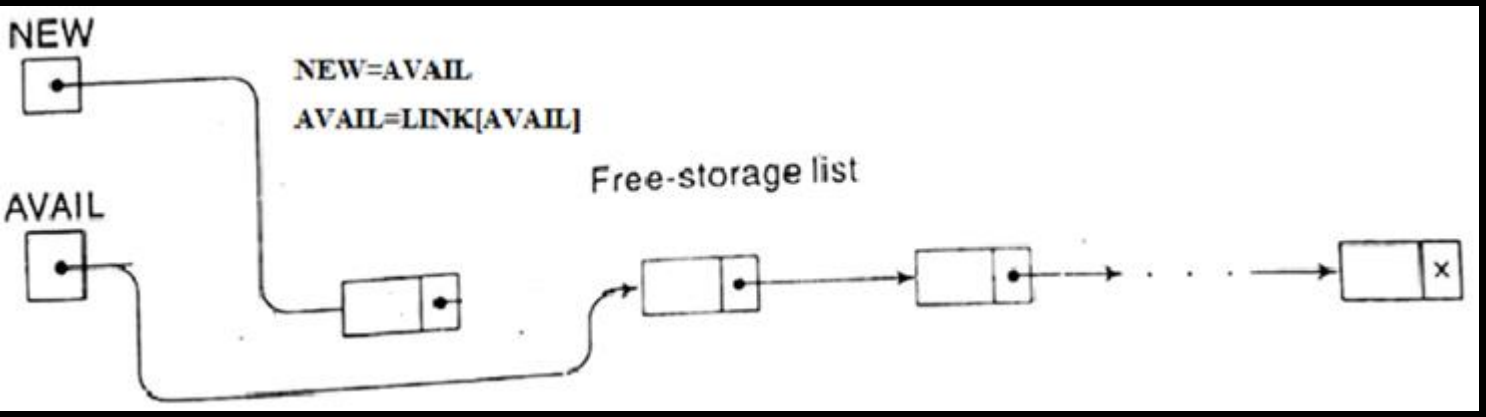




Inserting an element from t

1. **Inserting in an empty list**

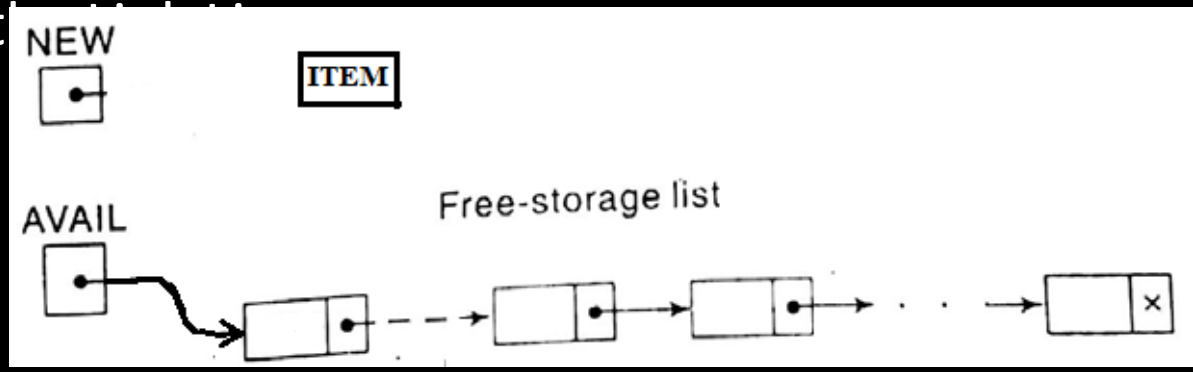


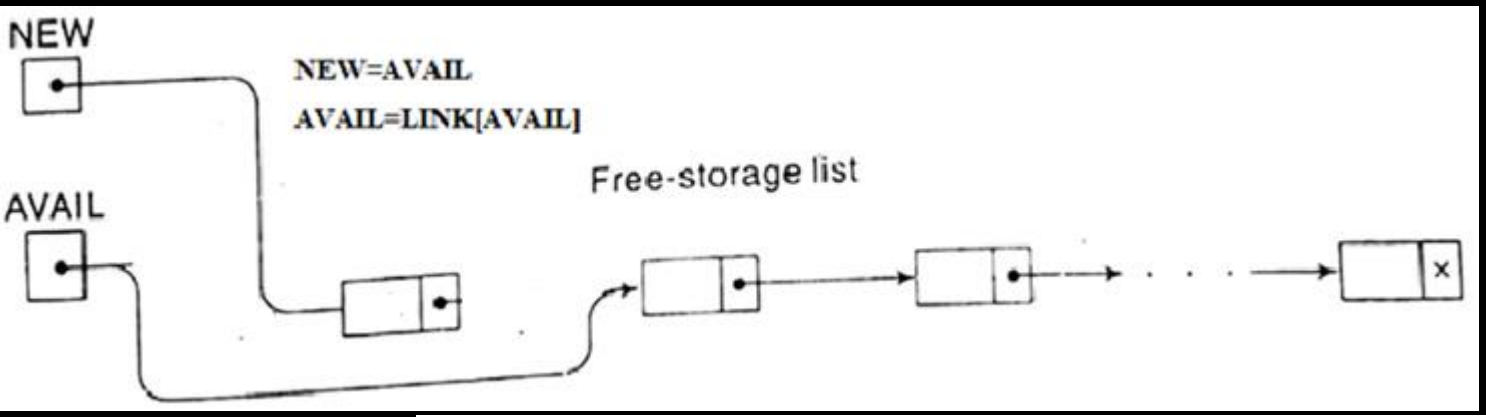


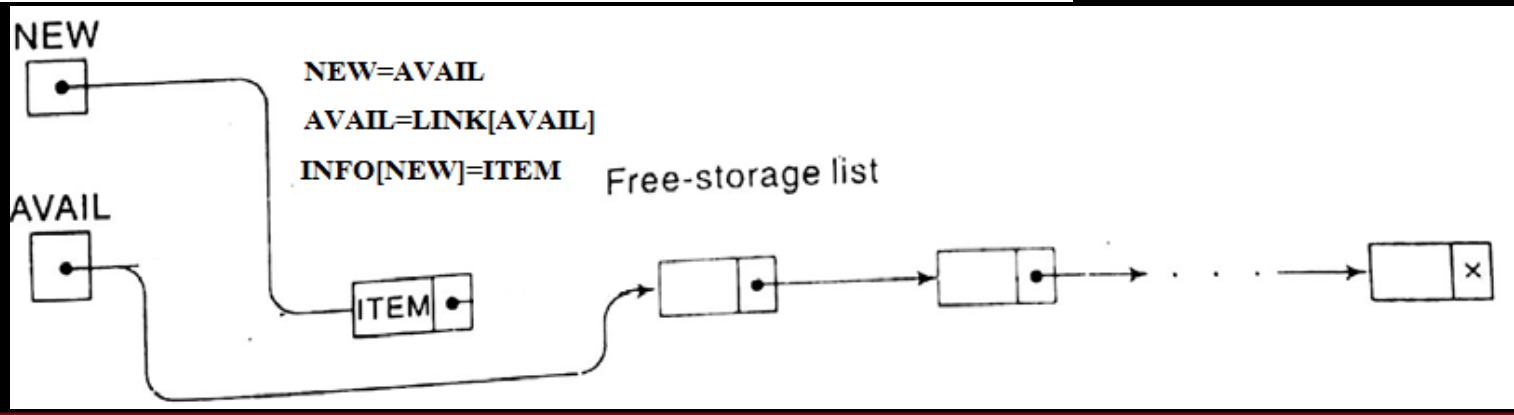
After removing dash lines, it looks like the left figure

Inserting an element from t

1. **Inserting in an empty list**



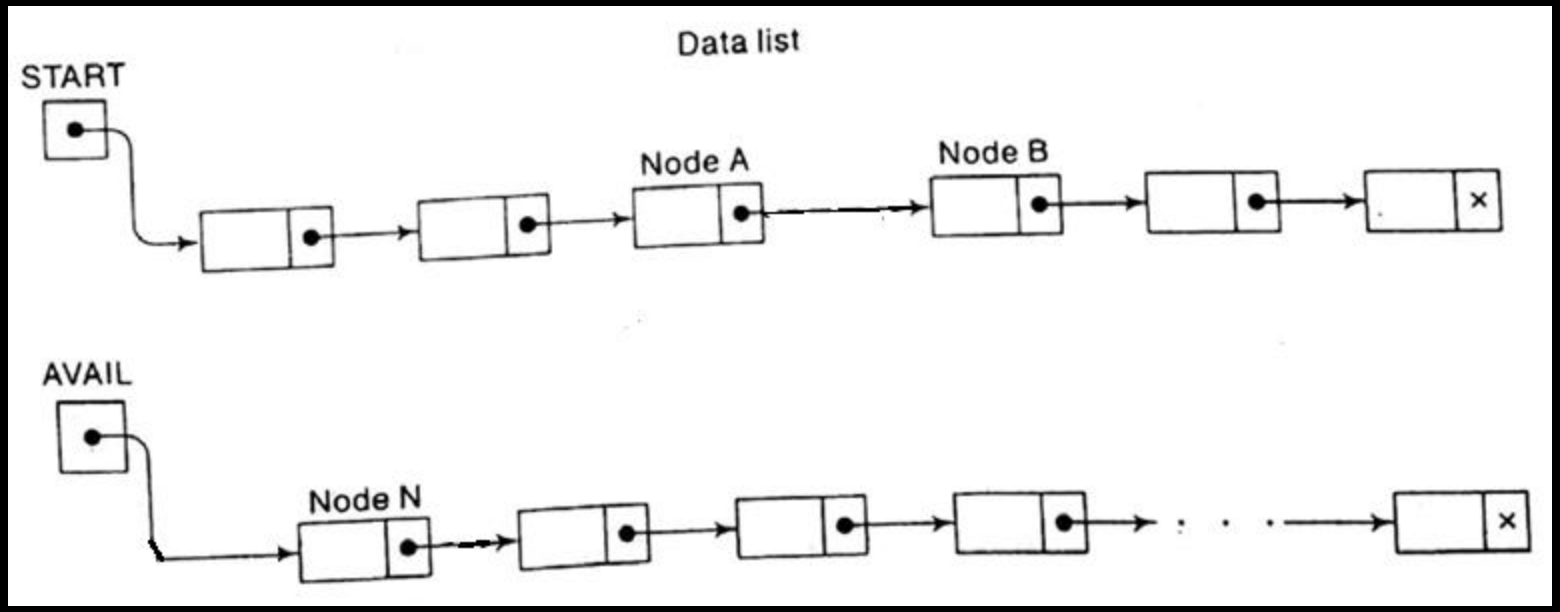




Inserting an element from the Link List

4. Inserting an ITEM to a given position

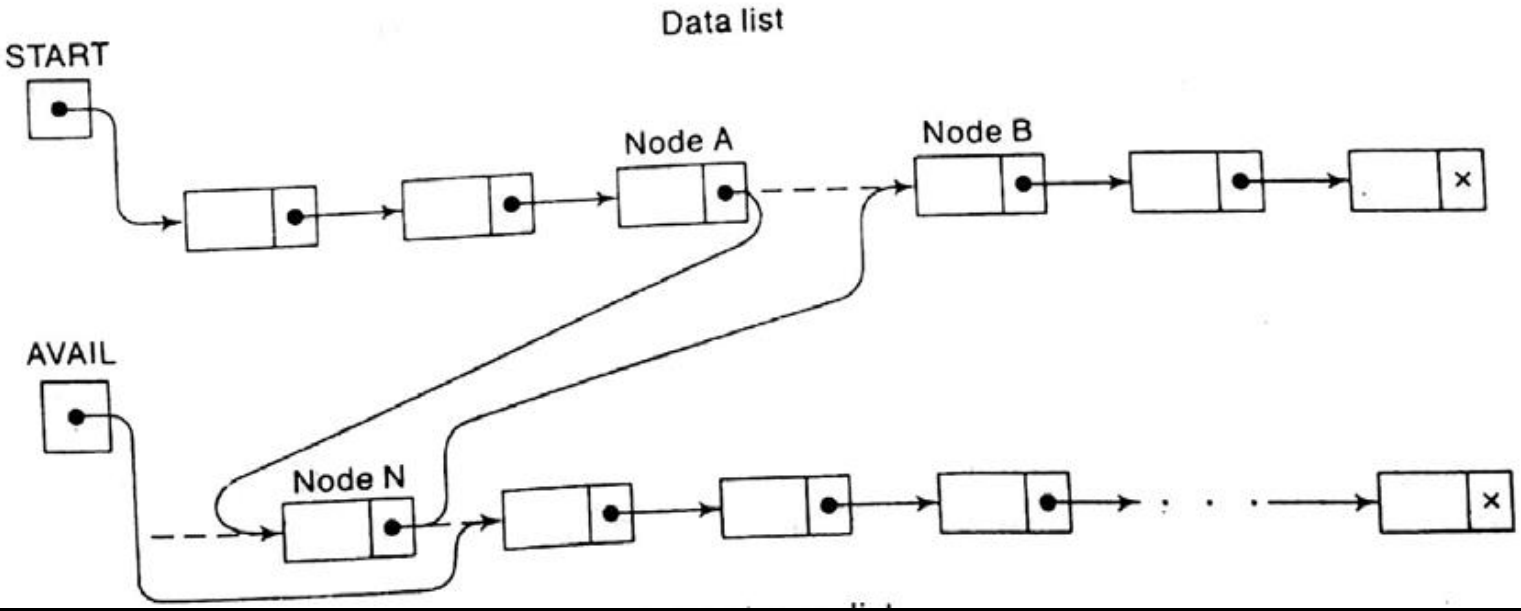
Now we want to insert a node after A and before B.



Inserting an element from the Link List

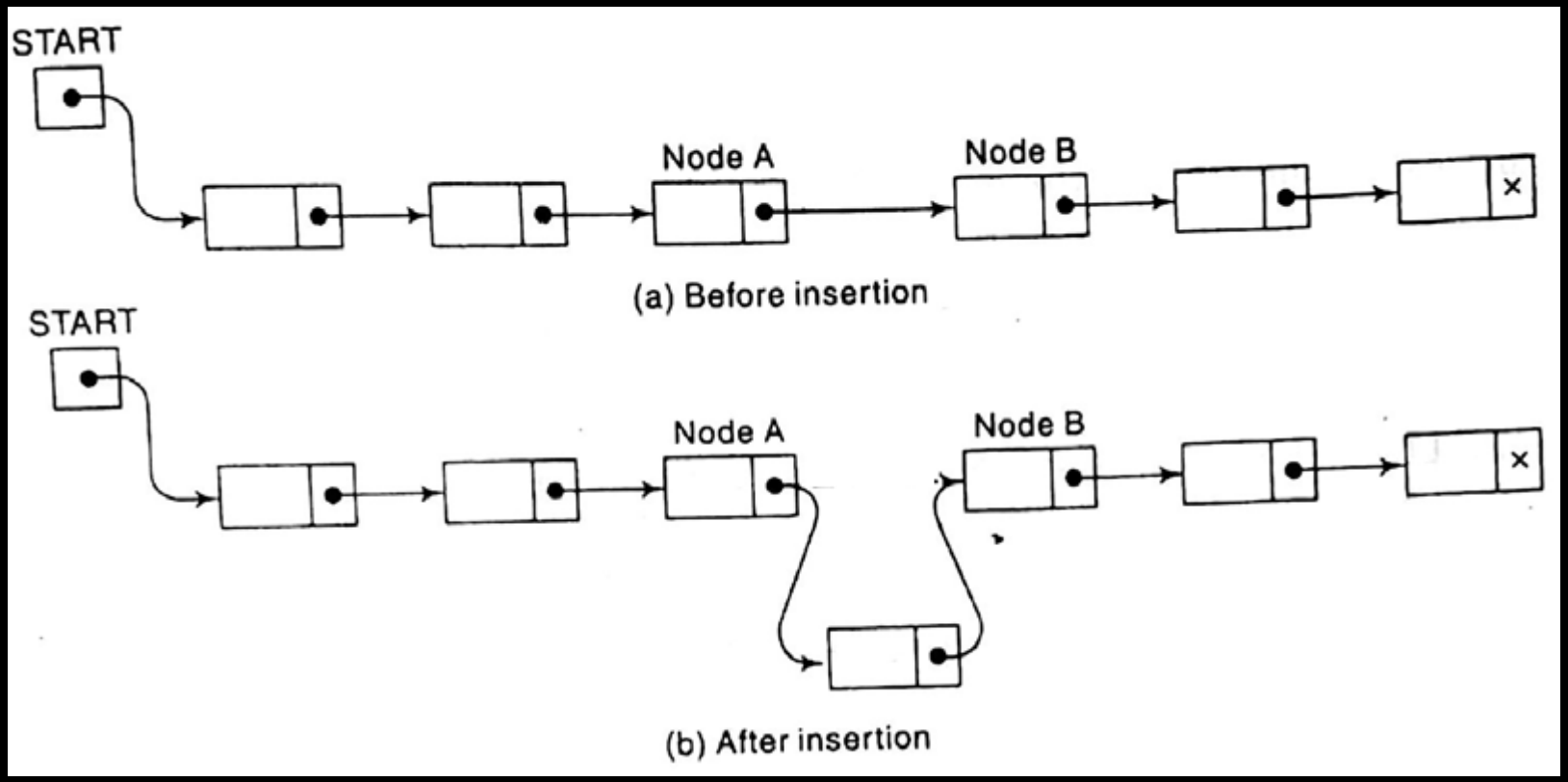
4. Inserting an ITEM to a given position

The nextpointer field of node A now points to the new node N, to which AVAIL previously pointed.



Inserting an element from the Link List

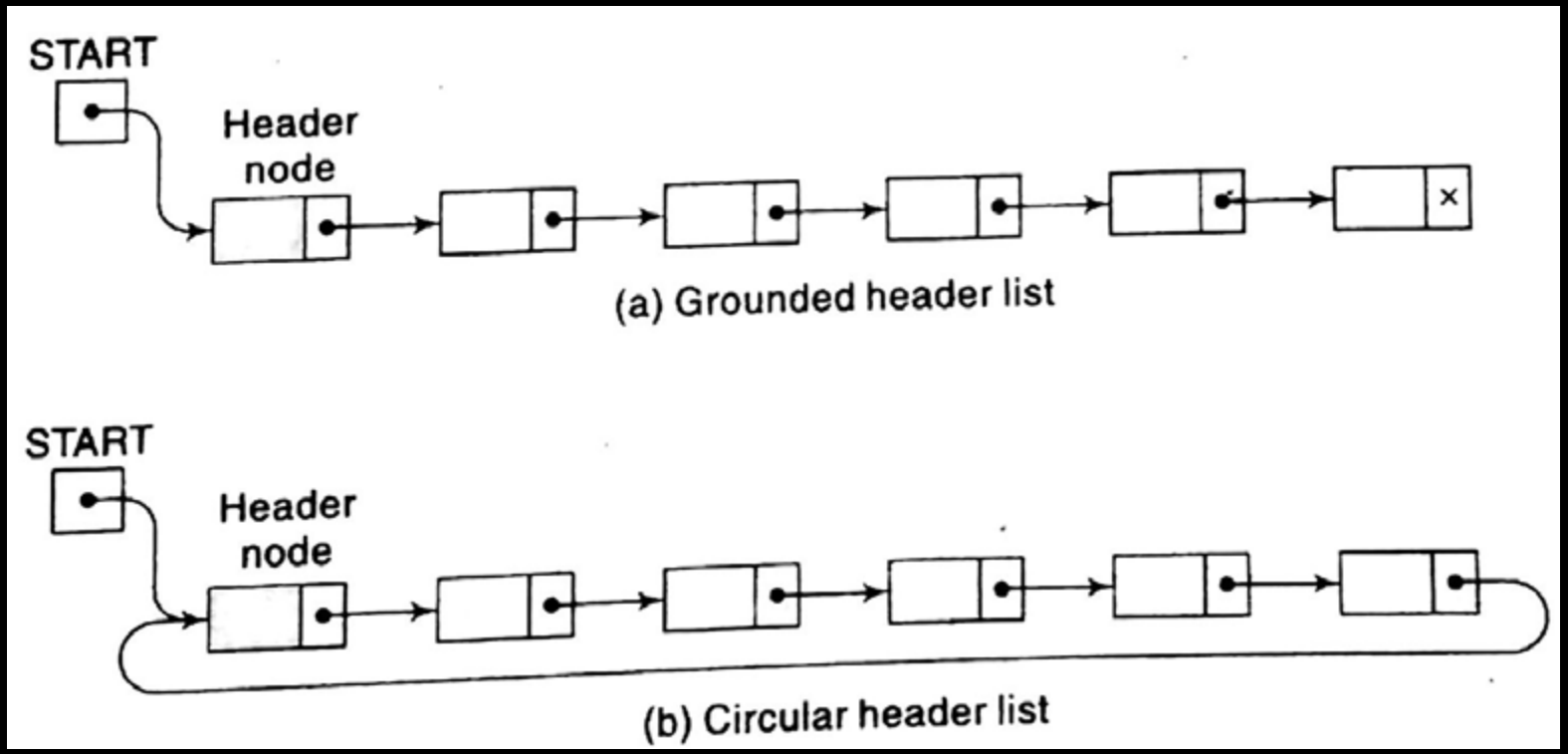
4. Inserting an ITEM to a given position



Header Link List

Contain a special node called header node at the beginning

1. Grounded header list
2. Circular header list



CSE 203: Data Structure

Data Structure

Two-way List

A *two-way list* is a linear collection of data elements, called *nodes*, where each node N is divided into three parts:

(1) An information field INFO which contains the data of N
(2) A pointer field FORW which contains the location of the next node in the list
(3) A pointer field BACK which contains the location of the preceding node in the list

The list also requires two list pointer variables: FIRST, which